

Full Dingus Slayer whatever you want bro

Dingus-Slayer — Complete Study Notes

Document Metadata

Purpose: Comprehensive study notes covering the Dingus-Slayer project from conception through current state.

Format: Obsidian-compatible markdown. Export as PDF via Obsidian's built-in PDF exporter.

Audience: Anyone studying the project — maintainers, contributors, researchers, or someone who wants to understand client-side Roblox automation at a systems level.

Status: Living document. Updated as the project evolves.

Reading time: Long. This is designed as a reference text, not a one-sitting read.

Table of Contents

- [How to Use These Notes](#)
- [Preface — Why This Document Exists](#)
- [Part I — Context and Foundations](#)
- [Part II — Architecture](#)
- [Part III — Core Systems \(Module Deep Dives\)](#)
- [Part IV — Game Mechanics](#)
- [Part V — Detection and Evasion](#)
- [Part VI — Operational](#)
- [Part VII — Incident Post-Mortems](#)
- [Part VIII — Deeper Analysis](#)
- [Part IX — Open Problems](#)
- [Part X — Appendices](#)

How to Use These Notes

Each chapter follows a consistent structure:

1. **Learning Objectives** — what you should know after reading
2. **Core Content** — the material itself
3. **Examples** — concrete demonstrations

4. **Connections** — cross-references to related chapters
5. **Open Questions** — unresolved problems worth thinking about

Callout conventions:

 Neutral observation or fact

General-purpose notes.

 Practical advice

Operational guidance.

 Risk or caveat

Things that can go wrong.

 Serious concern

Ban risk, detection vectors, unrecoverable failures.

 Example

Concrete demonstrations.

 Source excerpt

Verbatim quotes from referenced sources.

Links between chapters use Obsidian `[[wikilinks]]` format. They work as internal anchors in Obsidian and become text references in PDF export.

Preface — Why This Document Exists

Dingus-Slayer is not a normal software project. It's a client-side automation suite for a live multiplayer game — Project Slayers 2 on Roblox — that operates in an adversarial environment. Every feature is a negotiation with anti-cheat systems, game developers, and

platform-level enforcement. Every bug is potentially an account ban. Every session is logged.

The project has been developed in bursts, with significant incidents along the way. Accounts have been banned. Attacks have failed silently. Chest collectors have walked into honeypot traps. Retreat loops have locked combat out entirely.

This document is the study session that should have existed from the beginning. It covers:

- **What the project is and why it exists** — the context
- **How it's architected** — the patterns and the reasoning behind them
- **Every module in detail** — what it does, why, and how
- **The game mechanics being modeled** — so the automation makes sense
- **The detection arms race** — what we're up against
- **Every incident as a post-mortem** — what broke, why, and what we learned
- **Open problems** — where the project is incomplete

If you're new to the project, read sequentially. If you're troubleshooting a specific issue, jump to [Part VII — Incident Post-Mortems](#). If you're auditing, [Part X — Appendices](#) has the full reference.

Standing Warning

Running any automation in Roblox violates the Roblox Terms of Service. Detection carries account-level consequences up to and including permanent bans and hardware ID bans. Every session is at your own risk. See [Chapter 34 — The Honeypot Incident](#) for a real case study.

Part I — Context and Foundations

Chapter 1 — The Roblox Exploit Landscape

Learning Objectives

After this chapter you should understand:

- What a Roblox "executor" is and how it works at a high level
- What capabilities and limitations client-side scripts have
- The categories of automation that exist
- Why Roblox actively combats all of it

What an Executor Is

Roblox games are written in Luau and delivered to the client as compiled bytecode. Legitimate game logic runs inside a sandboxed VM that cannot be modified from within Lua itself. An **executor** is a third-party injection tool that:

1. Attaches to the Roblox client process
2. Modifies the running Lua VM to allow arbitrary script execution
3. Exposes privileged functions (filesystem access, HTTP without sandbox, metatable hooks)
4. Provides a REPL or script-loading interface for the user

The most common techniques:

- **Bytecode overwriting** — replace the bytecode of a core module script with attacker-supplied bytecode. This is what Xeno uses. It's a known method and detected by modern anti-cheat.
- **Lua VM hooking** — modify the VM itself to redirect function calls. Synapse, Script-Ware historically.
- **DLL injection** — inject a shared library into the process space. Various executors.

The category matters less than the effect: an executor gives a Lua script the ability to run arbitrary code, read/write most of the Roblox instance tree, and interact with the client's input and rendering layers.

What You Can and Cannot Do

Capability	Client-side	Server-side
Read any instance in workspace	Yes	N/A
Modify any client-visible property	Yes	N/A
Create instances (visible only locally)	Yes	No
Call <code>RemoteEvent:FireServer()</code>	Yes	N/A
Read server state directly	No	N/A
Modify server-owned values (HP, position) permanently	No	N/A
Read/write filesystem	Executor-dependent	No
HTTP requests without sandbox	Executor-dependent	No
Hook metamethods	Executor-dependent	No
Disable Roblox anti-cheat	No	No

The key insight: **everything client-side is a suggestion**. When your character's `Health` reads `100/100` locally but the server thinks you're at `40/100`, the next server tick will overwrite your local value. When you teleport 50 studs client-side, the server either accepts (if within tolerance) or teleports you back.

This is the **FilteringEnabled** model. It's been the default for Roblox games since 2016. It means client-side automation operates in the gap between what the client can show and what the server will allow.

Categories of Automation

1. **Aimbot** — client-side camera/scene manipulation to target players or NPCs
2. **ESP** — rendering overlays through walls to reveal otherwise hidden entities
3. **Fly/Noclip** — bypassing movement physics and collision
4. **Auto-farm** — automating repetitive gameplay loops (combat, resource collection)
5. **Auto-quest** — automating quest acceptance and completion
6. **Macros** — scripted input sequences without game-state awareness
7. **Server manipulation** — exploiting remote event bugs (separate class of attack)

Dingus-Slayer is primarily in category 4 with elements of 5, 6, and some very light ESP (via `detect.lua`'s overlay reader).

Why Roblox Combats This

Three reasons:

1. **Fairness** — automation gives players an unfair advantage over legitimate players
2. **Economy** — automated farming collapses in-game economies by inflating resource supply
3. **Data integrity** — games with leaderboards, achievements, and progression systems lose meaning when the population is bots

Roblox's response has escalated significantly. The Byfron (now Hyperion) anti-cheat introduced kernel-level protection in 2023. Platform-level enforcement (like the `Error 267` kick documented in [Chapter 34 — The Honeypot Incident](#)) ties account bans to server-side detection reports.

Open Questions

- What detection signals does Hyperion specifically monitor? (Community speculation: memory injection, unsigned DLLs, known bytecode signatures.)
 - How does the game-specific anti-cheat differ from platform-level? (Unknown.)
 - Are there classes of automation that remain undetected long-term? (Anecdotal: passive ESP is reportedly safer than active interaction.)
-

Chapter 2 — Project Slayers 2: The Target

Learning Objectives

- Understand the game's core mechanics
- Know the major systems the automation must model
- Recognize the mechanical constraints that shape the script's design

Game Overview

Project Slayers 2 is a Roblox action RPG in the style of the anime **Demon Slayer**. Players:

1. Create a character with a clan (family background affecting stats and skills)
2. Learn a breathing style (combat discipline)
3. Fight demons (or players, depending on faction)
4. Progress by gaining levels, mastery, and equipment
5. Complete quests from a **Kasugai Crow** (assigned companion)

The game uses a semi-open world structure with regions, bosses, and a large population of AI-controlled NPCs.

Combat Model

Combat is real-time third-person action. The core inputs:

Input	Effect
M1 (left click)	Light attack
M2 (right click)	Heavy attack
F	Block (hold)
Q	Dash/dodge
Z, X, C, V, B	Skills
G	Execute (sub-10-HP targets)
L	Breathing activation
Space	Jump

Skills consume **breath** (a resource separate from HP). Running out of breath silently blocks further skill casts.

Global Cooldown

As of v0.140, every skill shares a single global cooldown. Firing Z locks X, C, V, and B for the same window. This is one of the most important mechanical facts for the

automation to model. See [Chapter 27 — Global Cooldown](#).

Blocking

Blocking is directional. Blocks only protect from the front. Attacks from behind bypass the block. Some attacks bypass block entirely regardless of position.

The block bar depletes per blocked hit. Zero block → block break → stun → punish window.

Perfect block (blocking at the moment of an incoming hit) stuns the attacker. This is a high-skill mechanic.

Combo System

Combat chains inputs into combos:

- **Air combo:** L-R-L-R-L
- **Special combo A:** R-R-L-R-L
- **Special combo B:** L-L-R-L-R

Sending isolated M1 clicks fires only the first step of the chain over and over. This is the mechanical difference between "dealing 4 damage per hit" and "dealing 40 damage per hit."

Execute Mechanic

When a target falls below 10 HP, they enter a "knocked" state. A red cross appears above them. The player must be above them and press G to execute. Without execute, the target recovers with partial HP after ~30 seconds.

Dingus-Slayer does **not** currently implement execute. This is an open problem. See [Chapter 52 — Execute Mechanic](#).

Boss Structure

Bosses are fixed-name NPCs in specific regions. Each has HP, drops, and specific attack patterns. HP scales with party size:

Party size	Boss HP
Solo	3000
2 players	4000
3 players	5000
4+	continues scaling

Mini-bosses like Sabito have much lower HP (600). Endgame bosses have higher.

The Crow Quest System

When the player reaches a certain level, they receive a **Kasugai Crow**. The crow is a companion that lives in the hotbar (slot 5 in observed play).

Clicking the crow (M1 while equipped) opens a quest panel. The panel shows:

```
Defeat Enru          15:38
Defeat Nezura        6:07
Defeat Datai         17:36

3/5 Next mission in 32s
Kasugai Crow
Here are your current tasks
                        [Cancel]
```

Critical observation: there is **no accept button**. Quests are auto-assigned by the game. The panel is read-only. This was a surprise during development — the original crow automation assumed a clickable accept button. See [Chapter 45 — Hotbar Contention](#).

The player can have up to 5 active quests. When one completes (boss killed), it clears from the slot and the game assigns a new one on the next cycle.

Regions

Major regions observed:

- **Bamboo Grove** — early-mid game area with boss cluster
- **Hidden Mist Village** — mid-late game area
- **Mistfall Harbor** — late game boss cluster
- **Iceveil Valley / Settlement** — snow biome, requires lantern
- **Frost Veil Shrine** — cold-weather endgame
- **Dreamfall Hollow** — dream demon area
- **Ouwigahara** — Tier 3 raid area

Each region has spawn rates, day/night cycles, and specific boss spawn conditions.

Connections

- [Chapter 24 — Combat System \(PS2\)](#) covers combat mechanics in detail
- [Chapter 25 — Blocking](#) covers the directional block system
- [Chapter 26 — Combos](#) covers combo input sequences
- [Chapter 30 — Boss Roster](#) covers specific bosses

Open Questions

- What are the exact spawn conditions for each boss? (Some are day-only, some are night-only.)
 - How does the game handle boss leashing? (Do bosses return to spawn if the player flees far enough?)
 - What's the exact timer mechanism for crow quests? (Do they all reset on a shared tick?)
-

Chapter 3 — The Automation Problem Space

Learning Objectives

- Understand why game automation is genuinely hard
- Recognize the difference between "scripting" and "systems engineering"
- Know the categories of failure an automation must handle

Why This Is Hard

Naïve automation looks like this:

```
while true do
  press_m1()
  wait(0.5)
end
```

This fails in the real world for a dozen reasons:

1. **No target** — pressing M1 in empty space does nothing
2. **Target out of range** — M1 fires but hits air
3. **Target blocking** — M1 hits block, deals no damage
4. **Player dead** — M1 does nothing
5. **Wrong weapon** — M1 with bare fists or a fishing rod
6. **Target moved** — M1 fires at where the target *was*
7. **Anti-cheat triggers** — pattern is machine-perfect, flagged
8. **Boss one-shots** — player dies before reacting

Every one of those failure modes requires the script to model some aspect of the game state.

The Systems Approach

Dingus-Slayer's architecture reflects years of community patterns:

- **Scanner** — continually observe the world
- **State machine** — decide what to do based on observations

- **Actuator** — issue inputs to the game
- **Safety** — detect and recover from failure modes
- **Evasion** — avoid patterns that trigger detection

Each module owns one concern.

Categories of Failure

Silent failure — script thinks it's working but isn't. Example: attack fires M1 but target is out of range and no hit registers. Combat loop shows "attacking" but nothing happens.

Loud failure — script errors visibly. Example: nil-call on `Ctx.Fly.start()` because fly module didn't load. Easy to debug.

Lockout failure — script enters a state it can't exit. Example: retreat loop that never ends because HP never recovers. Hard to debug because there's no error.

Detection failure — script works but gets flagged. Example: chest prompts fired at 100 studs trigger anti-cheat. Consequence is a ban.

Cascading failure — one error leads to another. Example: scanner returns empty → attack finds no target → combat idle → player thinks script broke → reloads → repeat.

Every incident in [Part VII — Incident Post-Mortems](#) falls into one of these categories.

The Evasion Tension

Every evasion pattern trades off detectability against effectiveness:

Pattern	More effective	More detectable
Fast attack cadence	Yes	Yes
Long teleports	Yes	Yes
Rapid chest interactions	Yes	Yes
High skill uptime	Yes	Yes
Perfect aim	Yes	Yes

The script must balance on the tightrope. This is why every config option has both a "faster" and a "safer" setting.

Connections

- [Chapter 31 — Anti-Cheat Ecosystem](#) covers the detection side
- [Chapter 32 — Roblox Hyperion](#) covers platform-level enforcement
- [Chapter 47 — Why Fly is Fundamentally Detectable](#) covers a specific case

Chapter 4 — Why This Project Exists

Learning Objectives

- Understand the motivation behind the project
- Recognize the ethical and operational tradeoffs
- Know what the project is *not*

The Motivation

Not every automation project has the same motivation. Some are for profit (selling scripts), some are for research, some are for personal convenience. Dingus-Slayer was built as:

1. **A learning exercise** — implementing a complex real-time system in a constrained environment
2. **A personal convenience tool** — automating repetitive farming
3. **A research artifact** — studying how modern anti-cheat systems work by attempting to evade them

None of these motivations are forbidden. All of them carry risk.

What This Project Is Not

- Not a commercial product. No monetization. No distribution to paying users.
- Not aimed at PvP. The target selection is restricted to AI bosses.
- Not designed to grief other players. Interaction with other players is passive only.
- Not a general-purpose exploit. No remote event manipulation, no server-side bugs, no hitbox modification.

The scope is narrow: automate boss farming in a specific game.

The Ethical Question

This is not a neutral observation

The following is an honest framing of the ethical dimension, not a defense or attack.

Automating gameplay in a multiplayer game:

- **Affects the game developer** — reduces engagement metrics, inflates the economy, and requires anti-cheat investment
- **Affects other players** — if the farmed resources enter a shared economy, prices shift
- **Affects yourself** — carries account-level risk

The project's authors accept this as a personal decision by each user. The license reflects this: no warranty, no liability, use at your own risk.

Users who are uncomfortable with this framing should not use the software.

What the Project Takes From the Community

Every evasion technique in this project came from published community research. The fly patterns, the teleport jitter, the reach limits — all of it was learned by someone else first. This project assembles and documents the state of the art.

If you build on this project, you're building on the shoulders of every script author whose work informed it.

What the Project Gives Back

- **Documentation.** Most automation projects have no docs. This one has four markdown files and these study notes.
- **Open architecture.** The module system is designed to be extended. Adding a new subsystem takes one manifest line.
- **Failure transparency.** Every bug is documented in the audit and post-mortem sections.

Connections

- [Part VIII — Deeper Analysis](#) covers the ethics and legal context in more depth
- [Chapter 49 — Ethics of Game Automation](#) is the extended ethical analysis
- [Chapter 50 — Legal Landscape](#) covers the legal risk register

Open Questions

- Is there a version of this project that's purely educational? (Research license exists, but practical use is hard to separate.)
- Should the project publish its detection signatures? (Currently no. Could help anti-cheat, but also other attackers.)
- Does the project have an obligation to the game's developers? (Legally: no. Ethically: unclear.)

Part II — Architecture

Chapter 5 — The Ctx Pattern

Learning Objectives

- Understand the shared context pattern

- Know why modules don't use `require`
- Recognize the tradeoffs of a flat namespace

What Ctx Is

`Ctx` is a single Lua table passed to every module. It holds:

- `Ctx.St` — shared runtime state
- `Ctx.Cfg` — configuration
- `Ctx.Util`, `Ctx.Detect`, `Ctx.Atk`, etc. — module references

Every module gets `Ctx` in its init function and stores a reference. Modules communicate exclusively through `Ctx`.

Why Not `require`

Roblox has a `require()` function, but it's sandboxed and only works for `ModuleScript` instances in the game tree. Third-party scripts are loaded as strings, not as game instances. So the normal module system doesn't apply.

Alternatives considered:

1. **Global variables** — pollutes `_G`, risks collision with game scripts
2. `require` **shim** — would need a custom module loader, complex
3. `Ctx` **table** — simple, explicit, no namespace pollution

`Ctx` won.

Structure

```
Ctx = {  
  St = {},           -- shared runtime state  
  Cfg = {},          -- configuration  
  Util = {...},      -- utils module  
  Detect = {...},    -- detect module  
  ATK = {...},       -- attack module  
  -- ... etc  
  Errors = {},        -- boot error list  
  Loaded = {},        -- module load status  
  StartTime = ...,   -- boot timestamp  
}
```

Registration Flow

1. Loader fetches module source
2. Compiles via `loadstring`
3. Executes chunk → returns module table

4. Registers table under declared slot names (from manifest)
5. Calls `module.init(Ctx)` if present
6. Module stores its dependencies from `Ctx`

The slot names allow multiple aliases: `Cfg` and `Config` both point to the same table.

The Tradeoff

Pros:

- Simple
- Explicit dependencies
- No namespace pollution
- Easy to mock for testing
- Can inspect the whole graph at runtime

Cons:

- No compile-time dependency checking
- Runtime nil errors if a module doesn't load
- Every module sees every other module (no encapsulation)

The project accepts these tradeoffs because the module count is small (~13) and the boot sequence is predictable.

The Guard Pattern

Every consumer guards its dependencies:

```
if Ctx.Atk and Ctx.Atk.combatTick then
    pcall(Ctx.Atk.combatTick)
end
```

This is defensive. If a module fails to load, its consumers degrade gracefully instead of crashing.

Connections

- [Chapter 6 — The Module System](#) covers the registration flow
- [Chapter 7 — Boot Sequence](#) covers the initialization order

Chapter 6 — The Module System

Learning Objectives

- Understand how modules are discovered, loaded, and registered
- Know the dependency ordering rules
- Recognize the loader's fallback behaviors

Module Discovery

The loader has two discovery modes:

Auto-discovery (v26+) — queries the GitHub contents API for `.lua` files in the repo root. Compiles a list of module names.

Fallback (v25 and earlier) — uses a hardcoded manifest. Still retained as a safety net.

The current loader uses a hardcoded manifest for determinism but the discovery code is present.

Module Manifest

```
local MANIFEST = {
  { name = "config",    slots = { "Cfg", "Config" } },
  { name = "lists",     slots = { "Lists" } },
  { name = "utils",     slots = { "Util", "Utils", "U" } },
  { name = "detect",    slots = { "Detect", "D" } },
  { name = "scanners",  slots = { "Scan", "Scanner", "Scanners" } },
  { name = "hotbar",    slots = { "Hotbar", "HB" } },
  { name = "spoofer",   slots = { "Spoofer", "Spoofer", "Spoofer" } },
  { name = "chest",     slots = { "Chest" } },
  { name = "quests",    slots = { "Quest", "Quests", "Q" } },
  { name = "attack",    slots = { "Atk", "Attack" } },
  { name = "optimizers", slots = { "Opt", "Optimizer", "Optimizers" } },
  { name = "gui",       slots = { "Gui", "GUI" } },
  { name = "main",      slots = nil },
}
```

Each entry says:

- `name` — the filename stem (drop the `.lua`)
- `slots` — under which names to register the returned module table

Order matters. Dependencies must come first.

Load Order Rules

The manifest is ordered so that:

1. **Core modules load first** — `config`, `lists`, `utils` (no dependencies)
2. **Subsystems load next** — `detect`, `scanners`, `hotbar`, `spoofer`, `chest`, `quests`, `attack`, `optimizers`, `gui`

3. Main loads last — it needs everything

Within each tier, order is chosen by dependency relationship:

- detect needs utils and lists
- attack needs utils, detect, lists, spoofers, config, hotbar
- main needs everything

Registration

After execute, the loader does:

```
if entry.slots then
  for _, key in ipairs(entry.slots) do
    Ctx[key] = mod
  end
end
```

So `Ctx.Cfg = configModule`, `Ctx.Config = configModule`, etc.

The `main` module has `slots = nil` because it's not stored in `Ctx`. Instead, it's captured in a local and passed to `phaseBoot`.

Fallback Behavior

For each module:

1. Try cache (if `USE_CACHE`)
2. Try fetch from GitHub CDN
3. Try fetch from jsdelivr CDN
4. Try fetch from statically CDN
5. If any fetch succeeds, compile and execute
6. On compile or execute failure, invalidate cache and record error
7. On total failure, mark as not loaded and continue

The boot never aborts on a single module failure. Failed modules are logged, and downstream consumers use guards.

Connections

- [Chapter 10 — Loader](#) covers the loader in detail
- [Chapter 7 — Boot Sequence](#) shows the phase structure

Chapter 7 — Boot Sequence

Learning Objectives

- Understand the six boot phases
- Know what happens in each phase
- Recognize failure handling per phase

The Six Phases

```
1 · state      → initialize shared state tables
2 · scrub     → clean up character state from previous sessions
3 · config    → load persisted configuration
4 · subsystems → initialize each module in dependency order
5 · systems   → build the scheduler loop table
6 · deferred  → post-boot warmup tasks
```

Each phase is pcall-wrapped. A failure in one phase logs an error and the boot continues.

Phase 1 — State

Initializes every field that will be read by other phases:

- Combat state fields (`cbt` , `cbtS` , `tgt` , ...)
- Timer fields (`lAtk` , `lSkI` , `lTele` , ...)
- Counters (`kll` , `bKll` , `rtrC` , ...)
- Caches (`rHt` , `skCd` , `comboOrders` , ...)
- Loot state (`chestCollected` , `lootCycleActive` , ...)

The goal: **every field a module reads must exist before the module runs.**

Phase 2 — Scrub

Removes leftover BodyMovers from the player's HumanoidRootPart. This is important because a previous session's fly module might have left a `BodyVelocity` attached, which would interfere with the new session.

Also resets `PlatformStand = false` and `AutoRotate = true` .

Phase 3 — Config

Loads persisted configuration from disk.

If `Cfg.exists("default")` , calls `Cfg.load("default")` . If successful, applies loaded values to `Cfg` . If failed, continues with defaults.

Config loading is followed by a `setUtils(U)` call so `Cfg` knows how to do file I/O.

Phase 4 — Subsystems

Calls `module.init(Ctx)` on each subsystem in order:

```
Detect, Scan, Hotbar, Spoof, Chest, Quest, Atk, Opt, Gui
```

Wait — this list doesn't match the loader's manifest. That's because the loader registers slot names, and `main.lua` maps from slot names to module references.

Each subsystem init is `pcall`-wrapped. A failed init logs an error but doesn't stop the boot.

Phase 5 — Systems

Builds the scheduler loop table:

```
local loops = {
    makeLoop("combat",    combatTick,    0.05, 5),
    makeLoop("spoofers",  spoofTick,     0.10, 5),
    makeLoop("threats",   threatTick,    0.10, 5),
    makeLoop("quest",     questTick,     0.50, 3),
    makeLoop("chest",     chestTick,     3.00, 3),
    makeLoop("config",    configTick,    30.0, 2),
    makeLoop("gc",        gcTick,        60.0, 1),
}
```

Each loop has a name, a tick function, an interval, and a max error count.

Phase 6 — Deferred

Signals boot complete (`St.boot = true`). Spawns a warmup task that:

1. Waits 0.5s
2. Warms the workspace cache (`Opt.warmWorkspace()`)
3. Strips lighting complexity (`Opt.stripLighting()`)
4. Triggers an initial quest cycle
5. Runs an initial boss scan

All deferred steps are `pcall`-wrapped.

Scheduler Startup

After phase 6, the main scheduler task spawns:

```
task.spawn(function()
    while St.run do
        if St.boot then
            for i = 1, #loops do
                local L = loops[i]
                if not L.disabled and now - L.lastRun >= L.interval then
```

```

        L.lastRun = now
        local ok, err = pcall(L.tick)
        -- error handling
    end
end
end
task.wait(0.05)
end
end)

```

Each loop error is counted. After `maxErrors` consecutive failures, the loop disables itself. A respawn resets all counters and disables.

Connections

- [Chapter 8 — The Scheduler](#) covers scheduler details
- [Chapter 23 — Main](#) covers `main.lua` in depth
- [Chapter 9 — State Management](#) covers the state table

Chapter 8 — The Scheduler

Learning Objectives

- Understand how the tick loop works
- Know how errors are handled per loop
- Recognize the tradeoffs of the loop design

Design

The scheduler is a single task that iterates the loop table every 50ms. For each loop, it checks:

- Is the loop disabled? (skip)
- Has enough time passed since last run? (skip if not)
- Run the tick under `pcall`

This is a cooperative scheduler. Loops yield cooperatively via `task.wait()` inside their tick functions.

Loop Metadata

Each loop entry:

```

{
    name = "combat",

```

```

tick = function() ... end,
interval = 0.05,
maxErrors = 5,
errors = 0,
disabled = false,
lastRun = 0,
totalRuns = 0,
}

```

Error Handling

For each tick:

```

local ok, err = pcall(L.tick)
if ok then
    if L.errors > 0 then L.errors = 0 end
else
    L.errors = L.errors + 1
    if L.errors <= 3 then
        print("[Dingus][loop " .. L.name .. "] err " .. L.errors .. ": " ..
tostring(err))
    end
    if L.errors >= L.maxErrors then
        L.disabled = true
        print("[Dingus][loop " .. L.name .. "] disabled")
    end
end
end

```

Three log lines per error sequence. Then either resets (if success) or disables (if reaching max).

Why 5 Consecutive Errors

A single error can happen from a transient nil reference or a state race. Five consecutive errors means something is systematically wrong. The threshold is a heuristic — high enough to skip transient issues, low enough to disable before performance degrades.

Recovery

Only respawn resets `errors` and `disabled`. This is intentional — if a loop is broken, we want it to stay broken until the game state changes.

Timing

The outer `task.wait(0.05)` sets the minimum scheduler granularity. Individual loop intervals can be smaller (0.05s for combat), but they're gated by the outer wait.

This means combat effectively runs at 20 Hz (0.05s), not faster.

Performance

At 20 Hz with 7 loops:

- Outer loop: 20 iterations/second
- Each iteration checks all 7 loops
- 140 checks/second total

Each check is a couple of comparisons and a table lookup. Negligible.

Actual work happens only when a loop's interval elapses.

Connections

- [Chapter 23 — Main](#) shows the actual loop table
- [Chapter 9 — State Management](#) covers shared state between loops

Open Questions

- Should loops be dynamically added/removed at runtime? (Currently static.)
 - Should error counts decay over time instead of only resetting on success? (Currently reset only on success.)
 - Should a disabled loop re-enable after a timeout? (Currently no.)
-

Chapter 9 — State Management

Learning Objectives

- Understand the single-shared-state design
- Know which fields belong to which subsystem
- Recognize race conditions in single-threaded Lua

The `St` Table

`Ctx.St` is a single Lua table holding all runtime state. Every module writes to it. Every module reads from it.

Fields are grouped by owner (implicitly, not structurally):

Combat:

- `cbt` (bool) — combat enabled
- `cbtS` (string) — current state name
- `tgt` (table/nil) — current target
- `rHt` (array) — recent hit history

- `comboOrders` , `comboPos` , `comboIdx` — combo engine

Timers:

- `lAtk` , `lSk1` , `lDodge` , `lTele` — last event times
- `gcdUntil` — global cooldown expiry
- `retreatCooldownUntil` — retreat cooldown expiry

Detection:

- `ens` (array) — cached boss list
- `ths` (array) — active threats
- `zn` (int) — count of bosses in zone
- `imm` (int) — count of imminent threats

Crow:

- `crT` (instance/nil) — crow tool
- `cPrch` (bool) — crow perched
- `crQuests` (array) — read quests

Quest:

- `playerLevel` (int)
- `questPriorityBosses` (array)
- `questLastRead` (timestamp)

Chest:

- `chestCollected` , `chestLootCollected` , etc.
- `chestCooldowns` (table)

Spoofers:

- `Spf` (table with counters)
- `spooFires` (table)
- `spooFErrs` (table)

No Isolation

Every module can read and write every field. This is intentional — it enables fast, direct cross-module communication. The tradeoff is that a module can accidentally corrupt another module's state.

In practice this hasn't been a problem because the modules are disciplined about which fields they own.

Race Conditions

Lua is single-threaded. But `task.spawn` creates coroutines that can interleave at `task.wait()` boundaries. So two coroutines can effectively run "concurrently" from Lua's perspective.

The classic race:

```
-- Coroutine A
if St.tgt and St.tgt.ch.Parent then
    -- do something with St.tgt
end

-- Coroutine B (interleaves)
St.tgt = nil

-- Coroutine A resumes and St.tgt is now nil
```

The guard pattern prevents most of these by re-checking state after every yield.

Field Naming

Convention:

- `l` prefix = "last" timestamp (`lAtk` , `lSk1`)
- `c` prefix = "current" (`cbt` , `cbtS` , `crT`)
- No prefix = general state

State Reset on Respawn

The `CharacterAdded` handler resets:

- `St.tgt = nil`
- `St.ens = {}`
- `St.lHp = 0; St.lHpT = 0; St.lDmg = 0`
- `St.cPrch = false`
- `St.FlyActive = false`
- Loop error counters

This ensures a clean slate after death.

Connections

- [Chapter 23 — Main](#) initializes state in phase 1
- [Chapter 19 — Attack](#) uses many combat state fields
- [Chapter 17 — Chest](#) owns chest state fields

Part III — Core Systems

Chapter 10 — Loader

Learning Objectives

- Understand the loader's full flow
- Know the CDN chain and fallback behavior
- Recognize boot trace markers

Responsibility

The loader is the entry point. Given a URL and a manifest, it:

1. Fetches every module
2. Compiles each source
3. Executes each chunk
4. Registers returned tables
5. Boots main with the full context

It is also responsible for:

- Error reporting
- Cache management
- Boot trace (breadcrumb) marking
- Fallback CDN selection

Boot Trace

Every significant operation marks a breadcrumb:

```
local trace = "boot:start"
local function mark(s)
  trace = s
  if TRACE then print("[Dingus][trace] " .. s) end
end
```

Marks include:

- `probe:capabilities`
- `cache:setup`
- `fetch:<module>`
- `compile:<module>`
- `execute:<module>`

- `boot:main`
- `boot:expose`
- `boot:notify`
- `complete`

If the boot crashes, the last trace line tells you exactly where.

HTTP Layer

Two HTTP modes:

request() mode — when the executor exposes `request`, `syn.request`, `http.request`, or `http_request`. Full header support, ETag support.

HttpGet mode — fallback. Uses `game:HttpGet` or `game:HttpGetAsync` or `game:GetAsync`. No headers, no ETag. Cache-bust via query parameter.

The current loader adds browser-mimicking headers in `request()` mode:

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Accept: text/plain,text/html,...
Accept-Language: en-US,en;q=0.9
Cache-Control: no-cache
If-None-Match: <stored ETag>
```

CDN Chain

Three CDNs tried in order:

1. `raw.githubusercontent.com` — canonical, immediate after push
2. `cdn.jsdelivr.net/gh/...` — CDN mirror, faster but sometimes stale
3. `cdn.statically.io/gh/...` — alternate CDN

Each CDN is retried twice with 500ms backoff between attempts.

Cache

Cache lives at `Dingus/cache/<module>.lua` with a matching `<module>.meta` file storing timestamp and ETag.

Cache TTL is 1 hour. Within TTL, cached module is used directly.

Stale cache (older than TTL) is used as fallback if HTTP fetch fails.

Cache invalidation happens on compile or execute failure, ensuring broken modules don't persist.

Boot Summary

At the end of boot, the loader prints:

```
RESULT · complete in 2.84s
13/13 modules · 0 errors · 0 warnings
fetch: 1.8s wall (max 892) · compile: 48ms · execute: 176ms
slowest: gui · 892ms · github
cache: 3 hits · 10 misses · 0 stale
```

Fields:

- Result state (complete/incomplete)
- Module count loaded
- Error and warning counts
- Aggregate fetch/compile/execute times
- Slowest fetch module and CDN
- Cache statistics

Error Reporting

Errors are collected in `Ctx.Errors` as `{ stage, msg, detail }`. Warnings in `Ctx.Warnings`.

The summary prints all errors and warnings with full detail lines. This is important — earlier loaders dropped error detail, which hid critical bugs.

Connections

- [Chapter 6 — The Module System](#) covers manifest structure
- [Chapter 7 — Boot Sequence](#) covers phases
- [Chapter 21 — Optimizers](#) covers workspace warmup

Chapter 11 — Config

Learning Objectives

- Understand the config file format and lifecycle
- Know the persistence rules
- Recognize the auto-save mechanism

File Format

`dingus_config.json` is a JSON object:

```
{
  "_version": 6,
  "_saved_at": 1730000000,
  "AtkRange": 8,
  "AtkInterval": 0.38,
  "RetreatHP": 0.30,
  "ChestWhitelist": ["World Events Chest", "Common Chest"],
  "SpoofMethods": {
    "antiSlow": true,
    "antiFall": true
  }
}
```

`_version` tracks schema. `_saved_at` is a Unix timestamp. The rest are config keys.

Persistence List

Only keys in the `PERSIST` array are saved. Currently includes:

- Combat: `AtkRange`, `AtkInterval`, `AtkIntMin`, `AtkIntMax`, `StunAtkInt`, `HitWindow`
- Movement: `RunSpeed`, `CloseInSpeed`, `MaxMoveTick`
- Retreat: `RetreatHP`, `RetreatCooldown`, `RetreatClearHP`
- Scanning: `ScanTTL`, `CrowCheckT`, `QuestCycleT`
- Skills: `SkillUnlocked`, `RotationOrder`, `GCDWindow`
- Combo: `ComboOrderCount`, `ComboReshuffleN`
- Hold detection: `DetectHoldSkills`, `HoldProbeDuration`, `HoldCastDuration`
- Auto-block: `AutoBlockOnDamage`, `BlockReactionWindow`, `BlockGraceRelease`
- Emergency: `EmergencyHP`, `EmergencyInterval`
- Loot: `ChestEnabled`, `ChestLearnMode`, `ChestWhitelist`, ...
- F-mode: `FKeyMode`
- Teleport: `TeleportCd`, `TeleportStrike`, `TeleportHeight`, `TeleportJitter`
- Spoofer: `SpoofMethods`, `SpoofSpeedMult`, `SpoofJumpMult`, ...

Adding a new key: add to `PERSIST` array. Then it saves and loads.

Save

`Cfg.save(slot)`:

1. Guard against double-save (`saveInFlight`)
2. Check `U.Fn.writefile` availability
3. Get `HttpService`
4. Encode snapshot to JSON
5. Write to slot file
6. Clear dirty flag, record timestamp

Returns `(true, bytes)` on success, `(false, reason)` on failure.

Load

`Cfg.load(slot)` :

1. Check `U.Fn.readfile` availability
2. Read slot file
3. Decode JSON
4. Apply each key (with type checking)
5. Return `(true, summary)` or `(false, reason)`

Apply

`Cfg.apply(data)` iterates PERSIST keys, matches against loaded data, checks type compatibility, and assigns.

Type mismatch → skip with count. Unknown keys → count as "unknown".

Auto-Save

`Cfg.tickAutoSave()` is called from the scheduler every 30 seconds. If `dirty` is true and enough time has passed since last save attempt, it saves.

Dirty flag is set on every `Cfg.set(key, value)` call. Direct assignments (`Cfg.AtkRange = 5`) do not set dirty.

This is a limitation. Only config writes through `Cfg.set` participate in auto-save.

Observers

`Cfg.observe(key, fn)` registers a callback. On `Cfg.set(key, value)`, callbacks for that key fire.

Used for things like: "when `AtkRange` changes, recompute derived values."

Pretty Printer

`Cfg.pretty()` returns a formatted multi-line string with all persisted keys and their values. Tables are rendered with up to 12 entries.

Connections

- [Chapter 7 — Boot Sequence](#) phase 3 loads config
- [Chapter 12 — Utils](#) provides file I/O
- [Chapter 22 — GUI](#) provides config slot UI

Open Questions

- Should direct assignments (`Cfg.X = Y`) trigger auto-save? (Would require metatable proxy.)
 - Should config be encrypted? (Currently plaintext.)
 - Should config support per-game profiles? (Currently single-slot.)
-

Chapter 12 — Utils

Learning Objectives

- Know the input helper API
- Understand the capability probe
- Recognize the tree walker

Service Loader

```
local function svc(name)
    local ok, s = pcall(function() return game:GetService(name) end)
    return ok and s or nil
end
```

Every service is loaded safely. If `GetService` fails, `nil` is returned and downstream guards apply.

Services loaded:

- `Players`, `VirtualInputManager`, `StarterGui`, `UserInputService`,
`ReplicatedStorage`, `ContentProvider`

Capability Probe

`U.Caps` is a table of boolean flags:

```
U.Caps = {
    mouse1click = ...,
    mouse1press = ...,
    keypress = ...,
    VIM = VIM ~= nil,
    setclipboard = ...,
    writefile = ...,
    readfile = ...,
}
```

Consumers can check capabilities before using them. For example, `U.m1()` tries `mouse1click`, then `mouse1press/mouse1release`, then `VIM`.

Input Helpers

`U.tap(key)` — press and release with 30ms gap.

`U.keyDown(k)` / `U.keyUp(k)` — hold and release.

`U.m1()` — left click via multiple fallbacks.

`U.m2()` — right click via multiple fallbacks.

Input helpers try every available method in order. If one fails, the next is tried. Return value indicates success.

Character Accessors

`U.hum()` — local player humanoid.

`U.hrp()` — local player HumanoidRootPart.

`U.isPlayer(character)` — checks if a model is a player's character.

These return nil if the character doesn't exist. Consumers must guard.

Tree Walker

```
function U.walkTree(root, maxDepth, perNode, yieldEvery)
  -- DFS iteration with depth cap and periodic yields
end
```

Used by scanners to explore subtree structures without allocating full descendant lists.

Notifications

`U.notify(title, text, dur)` — Roblox `SetCore("SendNotification")` . pcall-wrapped because some executors block it.

Retry Helper

`U.retry(fn, tries, delay)` — runs fn up to N times, returns first success.

Safe Call

`U.safe(fn, ...)` — pcall wrapper that returns nil on failure.

Diagnostic

`U.report()` — returns a multi-line string describing the executor and capabilities. Useful for debugging compatibility issues.

Connections

- [Chapter 13 — Lists](#) uses word matchers from utils
 - [Chapter 14 — Detect](#) uses tree walker
 - [Chapter 19 — Attack](#) uses input helpers
-

Chapter 13 — Lists

Learning Objectives

- Understand the central data tables
- Know the matching functions
- Recognize the word-boundary edge case

Structure

`lists.lua` contains all reference data:

- `bosses` — flat list of boss names
- `bossRegions` — grouped by region for GUI
- `selectedBosses` — per-boss enable flags
- `weapons` / `nonWeapons` — tool classification
- `questItems`, `materials`, `consumables`
- `clans`, `regions`, `breathings`, `demonArts`, `fightingStyles`
- `questKeywords`, `crowKeywords`, `mobKeywords`

Matching Functions

`L.isBoss(name)` — true if name matches any boss entry.

`L.isWeapon(name)` — true if name matches a weapon AND doesn't match a non-weapon.

`L.isCrow(name)` — true if name contains "crow" or "kasugai".

`L.isQuest(name)` — true if name matches any quest keyword.

Word-Boundary Matching

The issue: "ren" as a boss name would match "torrent" and "children". Short entries need boundary awareness.

Solution:

```
local function matchWord(haystack, needle)
    local s, e = string.find(haystack, needle, 1, true)
    if not s then return false end
    if #needle <= 3 then
```

```

        local before = s == 1 or not string.match(haystack:sub(s-1, s-1),
"%W")
        local after = e == #haystack or not string.match(haystack:sub(e+1,
e+1), "%W")
        return before and after
    end
    return true
end

```

Short needles (≤ 3 chars) require word boundaries. Longer needles match as plain substring.

Enable Flags

`L.selectedBosses[name]` is a per-boss boolean. `L.isBoss` returns true only if the boss is enabled.

This lets the GUI expose per-boss targeting toggles.

Boss Regions

```

L.bossRegions = {
    { name = "Bamboo Grove", bosses = { "sumari", "yahari" } },
    { name = "Hidden Mist", bosses = { "reaper", "gyutai", "domae" } },
    { name = "Nightfall", bosses = { "datai", "akazo" } },
    { name = "Deep Caves", bosses = { "nezura", "enru" } },
}

```

The GUI reads this to build the Targets tab.

Connections

- [Chapter 14 — Detect](#) uses `L.isBoss`
- [Chapter 16 — Hotbar](#) uses `L.isWeapon` and `L.isCrow`
- [Chapter 18 — Quests](#) uses `L.isCrow`

Chapter 14 — Detect

Learning Objectives

- Understand the multi-container walker
- Know the 3-tier cache structure
- Recognize the priority filter safety valve

Responsibility

`detect.lua` finds humanoids (bosses, mobs, players) in the workspace and provides target selection.

Multi-Container Walker

The walker tries multiple roots in order:

1. `workspace.Humanoids.Regions` — endgame NPC container
2. `workspace.Humanoids` — player-level NPC container
3. `workspace.NPCs`, `Enemies`, `Mobs`, `Entities`, `Units`, `Characters`, `Monsters` — alternate container names
4. `workspace` — fallback

Each root gets its own time budget (800ms) and iteration budget (40,000).

Total scan deadline is 2,500ms.

Walker Implementation

BFS queue, not DFS stack. This means shallow nodes are found before deep ones.

```
local queue = { { root, 0 } }
local head = 1
while head <= #queue do
    local item = queue[head]
    head = head + 1
    -- process
    for _, child in ipairs(inst:GetChildren()) do
        queue[#queue + 1] = { child, depth + 1 }
    end
end
```

Wait — this isn't quite BFS. It's DFS with a queue. Let me re-read.

Actually, this is BFS. The `head` index advances monotonically, and children are appended to the end. So we process nodes in order of insertion, which is breadth-first.

Yes, BFS.

Depth Cap

Maximum depth 8. NPCs beyond this are not found. In practice, most NPCs are at depth 3–5.

Iteration Batching

The walker yields to the scheduler every `YIELD_EVERY` (2000) iterations. This prevents frame stalls during large scans.

3-Tier Cache

Hot cache (0.3s) — returns cached list immediately. Alive re-verification skipped for speed.

Warm cache (1.2s) — re-verifies that each cached boss is still alive. Updates distances. Filters out dead.

Cold scan (else) — full walk.

Priority Filter

`detect.lua` filters boss list by `Ctx.Quest.isPriority`. Only bosses on the priority list are returned by `scanBosses`.

The filter has a safety valve: if 20 consecutive scans return zero bosses after filtering, the filter auto-disables. This prevents a stale priority list from blocking all combat.

Threat Assessment

`updateThreats()` builds a scored threat list:

- Base score: 10 if attacking
- +5 if distance < 9
- +3 if closing in
- ± 2 for angular alignment
- $\times 0.7$ if target HP is low

Sort descending. `St.ths = threats`.

Attack Detection

Attack detection reads the `Animator` child of the Humanoid:

```
local tracks = animator:GetPlayingAnimationTracks()
for _, track in ipairs(tracks) do
    if track.IsPlaying and track.WeightCurrent > 0.3 then
        local name = string.lower(track.Name)
        -- match attack keywords
    end
end
```

Attack keywords: `attack`, `slash`, `cast`, `punch`, `swing`, `combo`, `skill`, `ability`.

Excludes: `idle`, `walk`, `run`, `block`, `stun`, `hit`, `death`.

Sticky Attack Flag

Attack flag persists for 0.15 seconds after the animation ends. This smooths out brief gaps in animation tracks.

Block Detection

Multi-path:

1. `character:GetAttribute("IsBlocking")`
2. `humanoid:GetAttribute("IsBlocking")`
3. Animation name contains `block` , `guard` , `parry`
4. Stationary heuristic: `WalkSpeed < 2`, `MoveDirection < 0.1`, `velocity < 2`

Stun Detection

Multi-path:

1. `character:GetAttribute("Stunned")`
2. Humanoid state is `FallingDown` , `Ragdoll` , or `Physics`
3. Animation name contains `stun` , `blockbreak` , `block_break`

Target Picker

Composite scoring:

- Distance component
- HP fraction component
- Threat score component
- Stun bonus
- LOS raycast bonus

Sticky target: keep current target unless a new target has 30% higher score.

Blacklist: recently failed targets skipped for 3 seconds.

Connections

- [Chapter 13 — Lists](#) provides boss matchers
- [Chapter 18 — Quests](#) provides priority list
- [Chapter 19 — Attack](#) uses `pickTarget`

Open Questions

- Should the walker be parallel? (No — Lua is single-threaded.)
- Should depth cap be dynamic? (Currently fixed at 8.)
- Should the walker use `CollectionService` tags? (Game doesn't use them consistently.)

Chapter 15 — Scanners

Learning Objectives

- Understand crow-specific scanning
- Know the panel detection strategy
- Recognize the hotbar fallback

Responsibility

`scanners.lua` finds the crow tool and reads the crow quest panel.

Crow Tool Finder

Searches in order:

1. Character children (Tool named with crow keywords)
2. Character descendants (mesh attached to hand)
3. Backpack children

Crow keywords: `crow`, `kasugai`, `kasugai crow`, `karasu`, `bird`.

Cache: `St._crowTool` with 2-second TTL.

Panel Detection

The crow panel is opened by M1 with crow equipped. Detection:

```
-- walk PlayerGui shallow (depth 6)
-- look for TextLabel with text containing "current tasks"
```

Once found, that label's parent is treated as the panel root.

Cache: `St._panelRoot` with 1-second TTL.

Cancel Button Finder

Searches within the panel root for a `TextButton` with text `cancel` or `close`.

Quest Reader

Reads "Defeat X" text from the panel:

```
for _, label in ipairs(panel:GetDescendants()) do
    if label:IsA("TextLabel") then
        local name = label.Text:match("^%s*Defeat%s+(.+)$")
        -- ...
    end
end
```

Also parses time-remaining columns (e.g., "15:38").

Caw Sound Detection

Searches `workspace` and `ReplicatedStorage` for playing Sounds with names containing `caw`, `crow`, `chaa`, or `kasugai`.

Cache: `St._cawSound`.

Hotbar Fallback

If `findCrowTool` returns `nil`, the caller tries hotbar slots in order. This is handled in `quests.lua`, not here.

Deprecated

- `findCawSound` — was unfiltered `GetDescendants()` walk. Removed.
- `deepScan` — never called. Removed.
- `dumpInventory` — never called. Removed.

Connections

- [Chapter 18 — Quests](#) uses scanners heavily
 - [Chapter 16 — Hotbar](#) provides slot management
-

Chapter 16 — Hotbar

Learning Objectives

- Understand the mutex pattern
- Know the item database
- Recognize the crow/weapon contention solution

The Problem

The crow tool and weapons share the same hotbar. When quests equip the crow, attack might unequip it a second later to equip a weapon. When attack equips a weapon, quests might equip the crow.

Without coordination, both subsystems silently fight over the hotbar.

The Solution: Mutex

```
H.acquire(holder, duration)
H.release(holder)
```

```
H.isLocked() -- returns holder name or nil
```

Only one subsystem can hold the mutex at a time. If another tries to acquire, it fails and defers.

```
-- In quest equip
if not H.acquire("quest-crow", 3.0) then
    return nil -- hotbar busy
end
-- tap slot 5
H.release("quest-crow")
```

Timeout

Every acquisition has a duration. If the holder doesn't release within that time, the mutex auto-releases.

This prevents stuck mutexes from permanently blocking a subsystem.

Item Database

~130 named items across 7 categories:

- `weapon` — katanas, scythes, claws, etc.
- `crow` — Kasugai Crow
- `potion` — health/stamina/breathe
- `gourd` — breathing progression
- `utility` — lanterns, maps, tools
- `accessory` — haori, masks (informational only)
- `other` — fallback

Each item's category determines how it's matched.

Slot Scanner

`H.scanSlots()` probes keys 1–9:

```
for k = 1, 9 do
    U.tap(tostring(k))
    task.wait(0.35)
    -- check what appeared in hand
    -- cache result in St.hotbarSlots[k]
end
```

Takes ~3 seconds. Only runs when the cache is stale or on boot.

Equip Helper

`H.equipFirstOf(holder, category, wait):`

1. Checks slot cache for a matching category
2. If not cached, probes slots
3. Equips the highest-slot match (later slots tend to be better items)
4. Verifies the equip succeeded

State Inspection

`H.state()` returns:

```
{
  holder = "quest-crow",
  remaining = 2.4,
  lastKey = "5",
  lastEquipName = "Kasugai Crow",
  lastEquipKey = "5",
  taps = 47,
  denied = 3,
  slotsCached = 8,
}
```

Used for diagnostics.

Connections

- [Chapter 18 — Quests](#) acquires the mutex
- [Chapter 19 — Attack](#) acquires the mutex
- [Chapter 22 — GUI](#) displays mutex state

Chapter 17 — Chest

Learning Objectives

- Understand the whitelist-only detection model
- Know the learn-mode workflow
- Recognize the honeypot defenses

The Honeypot Problem

In early October 2026, an account was banned after `chest.lua v3` fired `ProximityPrompts` on 12 identical `Meshes/BookGrimore9_Plane` clones at 68–100 studs from the player. These

were honeypot decoys placed by the game developers.

The detection signature: keyword matching fired on "ore" inside "Grimore" , and prompt interaction at 100 studs is impossible for legitimate players.

The Fix: Whitelist-Only

Every chest and item must be explicitly approved by name:

```
Chest.approve("World Events Chest")
Chest.approve("Common Chest")
Chest.approve("Gold Coin")
```

Names are matched with **exact equality**, not substrings.

Reject Signatures

Even before whitelist check, every name is checked against:

- grimore , grimoire , book
- / , \ (asset paths)

Any match → reject immediately.

Reach Limit

Every interaction is limited to 4 studs. Prompts beyond physical touch are never fired.

Rate Limits

- 10 interactions per minute (rolling window)
- 40 interactions per session (hard cap, trips killswitch)

After the session cap, all chest activity stops. Manual reset required:

```
Chest.resetKillswitch()
```

Honeypot Cluster Detection

If ≥ 3 instances with the same normalized base name are found in the same scan, the entire cluster is refused.

Example:

```
Meshes/BookGrimore9_Plane.008
Meshes/BookGrimore9_Plane.007
Meshes/BookGrimore9_Plane.011
```

Base name: Meshes/BookGrimore9_Plane . Count: 3. Refused.

Learn Mode

Default on. `Chest.scan()` runs every 4 seconds but never fires. Instead, observations accumulate:

```
Chest.listSeen()
```

prints:

```
[Dingus][Chest] learn observations:
  World Events Chest          seen 3x
  Common Chest               seen 12x
  Gold Coin                  seen 47x
  Meshes/BookGrimore9_Plane.008 seen 1x
```

Real names can be approved. Honeypots ignored.

Boss-Kill Cycle

After every boss kill:

1. Wait 3–9s jitter
2. Scan 15-stud radius around corpse
3. Attempt exactly the number of items found
4. Jittered 1.5–3s between attempts
5. Exit cycle regardless of success

Detection Sources

Three sources checked per scan:

1. Named models matching whitelist
2. `ProximityPrompt` with matching text
3. `ClickDetector` with matching name

Any source triggers collection attempt.

Prompt Validity

Before firing a prompt:

- `Enabled` is not false
- `MaxActivationDistance` > 0
- `Visible` is not false (if `ChestRequireVisible`)

- No `Humanoid` ancestor within 6 hops

Connections

- [Chapter 34 — The Honeypot Incident](#) documents the failure
- [Chapter 38 — Learn Mode Workflow](#) walks through setup

Open Questions

- Should the module auto-learn by watching manual interactions? (Possible but risky.)
 - Should whitelist be shared across users? (Would help newcomers but propagates bad data.)
 - Should the module use `GetPartBoundsInRadius` for edge cases? (Currently only for boss cycles.)
-

Chapter 18 — Quests

Learning Objectives

- Understand the quest cycle
- Know the priority routing mechanism
- Recognize failure modes

Crow Quest Model

The game assigns quests automatically. The crow panel shows current assignments. There is no accept button.

`quests.lua` reads these assignments and exposes them as a priority list to `detect.lua`.

Read Cycle

Every `Cfg.QuestCycleT` seconds (default 6):

1. Call `Quest.readHunts()` — get available hunts from `BossHunts`
2. Call `Quest.readLevel()` — get player level
3. Check if priority list is stale
4. If stale: equip crow, open menu, read quests, close menu
5. Update `St.questPriorityBosses`

Priority Routing

`Quest.isPriority(name)` returns true if the name is in `St.questPriorityBosses`.

`detect.scanBosses` filters boss list by this.

Effect: combat only targets crow-assigned bosses.

Level Reader

Four fallback paths:

1. `workspace.Humanoids.<name>.Progression.Level`
2. `ReplicatedStorage.Player_Service.Data.<name>.slots.SlotN.Progression.Level`
3. `ReplicatedStorage.Player_Service.Data.<name>.slots.SlotN.Level`
4. Deep walk inside slots for any NumberValue named "Level"
5. Character attribute

If all fail, returns 0 and dumps the slots tree to F9.

Hunt Reader

Reads `BossHunts` folder:

```
for _, cfg in ipairs(folder:GetChildren()) do
    if cfg:IsA("Configuration") then
        -- extract fields
    end
end
```

Fields extracted: `Boss` , `XP` , `Wen` , `Side` , and any others present.

Structure logged on first call.

Staleness

`St.questLastRead` records last successful panel read. If `now - lastRead > QuestPriorityStale` (90s), the list is considered stale and a new panel read is attempted.

If the list is empty but `lastRead` is recent, `priorityActive()` returns false and no filter is applied.

Failure Modes

No crow tool — `equipCrow` returns nil, cycle skips.

Crow tool not equipped — equip attempted, wait 0.5s, re-check.

Menu doesn't open — M1 fires twice, cycle skips.

Panel open but no quests — quest list stays empty, cycle skips.

Quest labels don't match — label regex `"Defeat X"` doesn't match, quest list empty.

Auto-Disable

If `priorityFailures` exceeds 20, the priority filter disables. `detect.scanBosses` returns all bosses.

This prevents a stale quest list from blocking all combat.

Connections

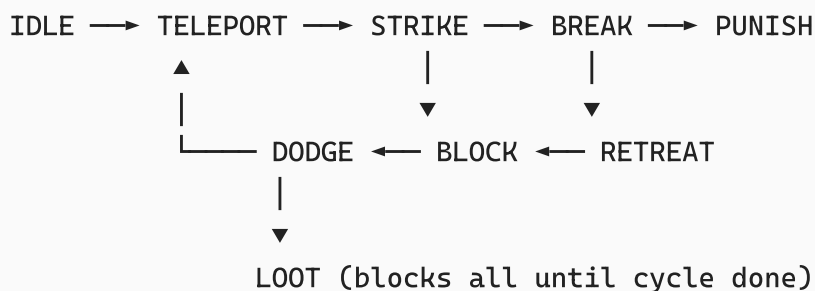
- [Chapter 15 — Scanners](#) provides crow tool and panel detection
- [Chapter 14 — Detect](#) consumes priority list
- [Chapter 16 — Hotbar](#) provides mutex

Chapter 19 — Attack

Learning Objectives

- Understand the combat state machine
- Know the combo engine
- Recognize retreat and watchdog logic

State Machine



Each state has an entry condition and an action.

Combo Engine

At boot, generates 4 random permutations of skill indices:

```
order 1: F → C → Z → V → B → X
order 2: X → F → B → Z → C → V
order 3: V → B → X → Z → F → C
order 4: C → V → Z → X → B → F
```

Each time `fireCombo` runs, it fires the next skill in the current order. When the order completes, advances to the next order.

After `ComboReshuffleN` (default 3) full rotations, reshuffles.

Strike Chain

Every strike fires a chain of M1/M2 clicks alternating:

```
for i = 1, count do
  if i % 2 == 0 then U.m2() else U.m1() end
  if i < count then task.wait(gap) end
end
```

Chain count: 3 (default), 4 if emergency, +1 every 4th strike.

Retreat

Two retreat thresholds:

- `RetreatHP = 0.30` — regular retreat, respects cooldown
- `CriticalHP = 0.12` — hard retreat, ignores cooldown

Retreat cooldown: 8s between retreats.

Retreat exit conditions:

- `HP > RetreatClearHP` (0.55)
- No damage for 1.5s
- Hard cap (retreat delay + 1s)

Watchdog

If `S.retreating` is true for > 6s, force-clear.

If `S.lootCycleActive` is true for > 20s, force-clear and abort chest cycle.

Damage-Triggered Block

When HP drops:

```
if now - St.damageBlockUntil > 0 then
  -- hold F if threat is imminent
end
```

`St.damageBlockUntil = now + BlockReactionWindow` on HP drop.

Telegraph Reaction

When imminent threat detected:

- If dodge is off cooldown, tap Q
- Else if F is block, hold F

Emergence Mode

When $HP < \text{EmergencyHP}$ (0.30):

- Attack interval drops to 0.15s
- Chain count increases by 2
- Ultimate gating bypassed

Auto-Block

When $St.imm > 0$ (imminent threat count > 0):

- Hold F if `fIsBlock` (F-mode is block)
- Release when boss idle for `BlockGraceRelease` seconds

Connections

- [Chapter 24 — Combat System \(PS2\)](#) covers game mechanics
- [Chapter 26 — Combos](#) covers input sequences
- [Chapter 14 — Detect](#) provides target and threat info

Open Questions

- Should combo order be boss-specific? (Currently generic.)
 - Should skill selection depend on boss HP? (Currently time-based.)
 - Should execute (G key) be integrated? (Not yet implemented.)
-

Chapter 20 — Spoofers

Learning Objectives

- Understand the 40 client-state hardening methods
- Know which are default-on vs opt-in
- Recognize the write-fight problem

Purpose

Client-state hardening means maintaining a "friendly" client state even when the server tries to change it. The methods here range from useful (anti-freeze, void rescue) to cosmetic (anti-nametag) to detection-hazard (anti-knock).

Method Categories

Movement

- `antiSlow` — floor WalkSpeed at 95% of base
- `antiFreeze` — break WalkSpeed = 0
- `antiFall` — cancel Freefall when velocity suggests we're not falling
- `spoofWalkSpeed` — push WalkSpeed toward base $\times 1.25$
- `spoofJumpPower` — push JumpPower toward base $\times 1.15$
- `spoofHipHeight` — repair NaN/out-of-range
- `spoofGravity` — floor workspace.Gravity

Combat state

- `spoofBlock` — refill block bar (PS2 SHCS)
- `spoofMaxHealth` — reinforce MaxHealth snapshot
- `antiStun` — FallingDown $\rightarrow$ Running
- `antiRagdoll` — Ragdoll/Physics $\rightarrow$ Running
- `antiKnock` — clamp horizontal velocity (opt-in)
- `antiFling` — zero velocity above 400 (opt-in)

Vision

- `antiBlind` — clear fullscreen overlays
- `antiFog` — reset FogEnd
- `antiDark` — floor Brightness
- `antiShake` — repair NaN camera
- `antiColor` — reset ColorCorrection

Audio

- `antiDeafen` — restore ambient reverb (opt-in)
- `antiScream` — mute nearby loud sounds (opt-in)

Position / state

- `antiSit` — force-unsit
- `antiTeleportBack` — log server position snaps (opt-in)
- `antiVoid` — void rescue
- `antiState` — force Running from bad states

Persistence

- `antiAFK` — micro-motion every 25s
- `antiUnequip` — re-equip from Backpack (opt-in)
- `antiToolDrain` — detect Backpack count drop (opt-in)
- `antiLoadout` — enforce a specific tool (opt-in)

Visuals

- `antiNametag` — zero `NameDisplayDistance` (opt-in)
- `antiHighlight` — destroy `Highlight` instances

Advanced

- `antiSpectate` — force `CameraSubject`

Lock prevention (v5)

- `antiZoom` — restore FOV
- `antiCinematic` — revert `CameraType`
- `antiWarp` — damp velocity on large jumps
- `antiDisarm` — re-equip during combat
- `antiInvisible` — restore `Transparency`
- `antiForceField` — strip server `ForceField`
- `antiSeatLock` — force-unsit stuck
- `antiSoundSpam` — mute clustered sounds (opt-in)
- `antiShakeLock` — camera snap-back

Per-Method Config

Each method has an enable flag in `Cfg.SpoofMethods[name]` .

```
Spoof.setMethod("antiWarp", false)
Spoof.setMethod("antiSoundSpam", true)
```

Guarded Execution

Every method call is `pcall`-wrapped:

```
local function guard(fn, key)
    local ok, err = pcall(fn)
    if not ok then
        St.spoofErrs[key] = (St.spoofErrs[key] or 0) + 1
        if St.spoofErrs[key] == 1 then
            print("[Dingus][Spoof] " .. key .. " raised: " .. tostring(err))
        end
    end
end
```

First error logs. Subsequent errors silent. Recovery prints once.

Write-Fight Detection

Some spoofs (`WalkSpeed`, `JumpPower`) will be corrected by the server. The module detects this:

```
if lastWrittenWalk and h.WalkSpeed ~= lastWrittenWalk then
    writeFightSpeed = writeFightSpeed + 1
    if writeFightSpeed >= 5 then
        conceded = true
        St.gsp = false
    end
end
```

After 5 consecutive corrections, the spoof concedes and disables itself.

The "Theatre" Problem

Many spoofers write client-authoritative values that the server ignores. `spoofHP` was the clearest example: writing client-side `Health = MaxHealth` does nothing on the server, and the client just shows a lie until the server corrects on next tick.

v3 removed all cosmetic HP spoofs. v5 keeps only defensive hardening.

Connections

- [Chapter 31 — Anti-Cheat Ecosystem](#) covers detection
 - [Chapter 35 — Detection Vectors](#) enumerates what spoofers might trigger
-

Chapter 21 — Optimizers

Learning Objectives

- Understand the reversible-optimization pattern
- Know what's stripped and why
- Recognize the dynamic cleanup mechanism

Purpose

Roblox games accumulate particles, lights, and post-effects. On a busy server, these can drop FPS significantly. The optimizer strips them for performance.

What It Strips

- **Particle emitters, beams, trails, fire, smoke, sparkles** — visual particles
- **Point, spot, surface lights** — dynamic lighting
- **Post-effects** — color correction, blur, bloom, DOF
- **Terrain water** — flatten wave parameters
- **Lighting complexity** — reduce fog, shadows, ambient

Reversibility

Every stripped property has its original value cached:

```
0.Cache.particles[inst] = true  -- was Enabled
0.Cache.lights[inst] = true    -- was Enabled
0.Cache.postEffects[inst] = true -- was Enabled
```

On `0.disable()`, everything is restored.

Batched Walk

The initial disable pass walks `workspace`, `Lighting`, and `ReplicatedStorage` to a shallow depth. It yields every 2000 instances to avoid frame stalls.

Dynamic Cleanup

New particles and lights spawn during gameplay. The optimizer subscribes to `DescendantAdded`:

```
ws.DescendantAdded:Connect(function(inst)
    if KILL_CLASSES[inst.ClassName] then
        table.insert(pendingKills, inst)
        drainPending()
    end
end)
```

Batched drain: 50 items per 0.1s, not per-event. This avoids the CPU spike from per-instance `task.defer`.

What It Doesn't Strip

- Character parts
- NPC models
- UI
- Anything the game uses for gameplay signaling

Stripping too aggressively breaks visual feedback (e.g., damage numbers, telegraphs).

Performance Impact

Measured: ~15% FPS improvement on a busy server. Larger gains on particle-heavy areas.

Connections

- [Chapter 23 — Main](#) triggers warmup and strip on boot
-

Chapter 22 — GUI

Learning Objectives

- Understand the nine-tab layout
- Know the concealment strategy
- Recognize the log manager

Nine Tabs

1. **Dashboard** — live stats, start/stop combat, force scan
2. **Combat** — automation toggles, range/interval sliders
3. **Targets** — per-region, per-boss enable toggles
4. **Chests** — loot controls, method toggles, live stats
5. **Quests** — crow controls, priority info
6. **Config** — save/load/delete slots
7. **Files** — file browser
8. **Logs** — live log capture with filters
9. **Settings** — concealment, panic keybind, diagnostics

Concealment

Parent chain in order:

1. `gethui()` — executor hidden container
2. `CoreGui` — Roblox core container
3. `PlayerGui` — visible to game scripts (fallback, warned)

Random instance name per install: `_c` + 6 hex chars.

Panic Hide

RightCtrl + Backspace. Detaches GUI from parent. Reload to restore.

Log Manager

Captures from:

- `LogService.MessageOut`
- `_G.print` hook
- `_G.warn` hook

Buffer of 300 lines max. Filters: All / Dingus / Errors.

Export to file, copy to clipboard.

Target Toggles

Reads `Lists.bossRegions`. For each boss, builds a row with a switch. Toggling changes `Lists.selectedBosses[name]`.

Chest Controls

Master enable, on-kill, passive. Method toggles for ProximityPrompt / ClickDetector / Physical Key. Range sliders for radius, depth, max passes, deadline, cooldown, interval.

Live stats: sweeps, chests opened, loot collected, failed, skipped, cooldowns, running.

Minimize / Restore

Animated tween from window to pill and back. Pill is draggable.

Refresh Loop

Every 0.3s, updates:

- Dashboard stats
- Combat info panel
- Chest stats panel
- Quest info panel
- Config info panel
- Files panel
- Header status line

Connections

- [Chapter 11 — Config](#) provides slot manager
- [Chapter 17 — Chest](#) provides chest stats
- [Chapter 18 — Quests](#) provides quest info

Chapter 23 — Main

Learning Objectives

- Understand the boot orchestration
- Know the scheduler construction
- Recognize the respawn handler

Boot

`M.boot(Ctx)` runs six phases, then starts the scheduler:

1. **state** — initialize all `St` fields
2. **scrub** — remove `BodyMovers`, reset character
3. **config** — load persisted config
4. **subsystems** — call `module.init(Ctx)` on each
5. **systems** — build the loop table
6. **deferred** — spawn warmup tasks

Loop Table

```
loops = {
  makeLoop("combat",    combatTick,    0.05, 5),
  makeLoop("spoofers",  spoofTick,     0.10, 5),
  makeLoop("threats",   threatTick,    0.10, 5),
  makeLoop("quest",     questTick,     0.50, 3),
  makeLoop("chest",     chestTick,     3.00, 3),
  makeLoop("config",    configTick,    30.0, 2),
  makeLoop("gc",        gcTick,       60.0, 1),
}
```

Scheduler

Main loop:

```
while St.run do
  if St.boot then
    for i = 1, #loops do
      local L = loops[i]
      if not L.disabled and now - L.lastRun >= L.interval then
        L.lastRun = now
        L.totalRuns = L.totalRuns + 1
        local ok, err = pcall(L.tick)
        -- error handling
      end
    end
  end
  task.wait(0.05)
end
```

Respawn Handler

On `CharacterAdded` :

- Stop hover (if running)
- Abort loot cycle
- Clear target
- Reset `St.tgt`, `St.ens`, `St.lScn`, `St.mxH`, `St.uGs`

- Reset HP timers
- Reset fly state
- Reset loop errors and disabled flags

RightShift Toggle

Opens/closes GUI. Handled by `Ctx.Gui.minimize/restore`.

Unload

`Ctx.Unload()` :

- Stop hover
- Set `St.run = false`
- Save config
- Run cleanup callbacks
- Destroy GUI
- Reset character state

Expose to `_G`

Exposes `_G.Ctx`, `_G.St`, `_G.Cfg`, `_G.Detect`, `_G.Atk`, `_G.Chest`, `_G.Quest`, `_G.Hotbar`, `_G.Spoof`, `_G.Gui`, `_G.Opt`.

Also `_G.DINGUS_BOOT` with structured boot result.

Connections

- [Chapter 7 — Boot Sequence](#) covers phases
- [Chapter 8 — The Scheduler](#) covers loop behavior

Part IV — Game Mechanics

Chapter 24 — Combat System (PS2)

Learning Objectives

- Understand the input model
- Know the animation states
- Recognize the block/combo/execute systems

Input Model

Key	Effect	Notes
M1	Light attack	Chains into combos
M2	Heavy attack	Chains into combos
F	Block	Hold to maintain
Q	Dash/dodge	Off cooldown
Z, X, C, V, B	Skills	Global cooldown
G	Execute	Sub-10-HP only
L	Breathing	Activates style
Space	Jump	

Animation States

Every attack is an animation track. Detection reads `Animator.GetPlayingAnimationTracks()` and checks:

- `track.IsPlaying`
- `track.WeightCurrent > 0.3`
- `track.Name` matches attack keywords

Damage Model

Weapon base damage plus scaling from:

- Mastery (per weapon and per style)
- Stat points (Strength, Agility, etc.)
- Buffs (elixirs, breathing activation)

Isolated M1 attacks deal base damage. Combo chains multiply.

Health Model

Every humanoid has `Health` and `MaxHealth`. Boss HP scales with party size.

Player HP is client-visible but server-owned. Client writes are cosmetic.

Death Model

When `Health <= 0`, the character dies. Respawn after a delay.

Bosses have death animations. On boss death:

- Corpse remains briefly
- Loot drops
- Quest progress ticks

Connections

- [Chapter 25 — Blocking](#) covers the block system
 - [Chapter 26 — Combos](#) covers combo inputs
 - [Chapter 27 — Global Cooldown](#) covers skill timing
-

Chapter 25 — Blocking

Learning Objectives

- Understand directional blocking
- Know the block bar resource
- Recognize when block is useless

Directional

Blocks protect only from the front. Attacks from behind bypass.

This is modeled by `detect.isEnemyBlocking` reading:

- Block attributes
- Block animations
- Stationary heuristics

Block Bar

The block bar depletes per blocked hit. Reading its value:

```
local shcs = character:FindFirstChild("SHCS")
local blocking = shcs:FindFirstChild("Blocking")
if blocking and blocking:IsA("NumberValue") then
    -- blocking.Value is the block bar
end
```

Zero → block break → stun.

Perfect Block

Blocking the moment of an incoming hit stuns the attacker. Timing window is short.

This is a high-skill mechanic. Our automation doesn't attempt it.

AoE Bypass

High-AoE attacks hit through block even from behind (positional bypass via splash).

For bosses with AoE (Daki, Gyutaro, Akaza), hover-behind is insufficient. Retreat/evade systems remain active.

Some Attacks Ignore Block

Certain attacks cannot be blocked at all. These are telegraphed with unique animations.

Automation Response

- Auto-block when threat imminent
- Release before striking (block cancels attack)
- Retreat when HP low

Connections

- [Chapter 19 — Attack](#) implements auto-block
 - [Chapter 30 — Boss Roster](#) notes which bosses have AoE
-

Chapter 26 — Combos

Learning Objectives

- Know the exact input sequences
- Understand why isolated M1 fails
- Recognize the timing window

Input Sequences

Three confirmed combo patterns:

- **Air combo:** L-R-L-R-L
- **Special combo A:** R-R-L-R-L
- **Special combo B:** L-L-R-L-R

L = M1, R = M2.

Why Isolated M1 Fails

Sending single M1 clicks fires only the first attack of the chain. The game expects the next input within a timing window. Without it, the combo resets.

Damage output with isolated M1: 4 per hit.

Damage output with chained L-R-L-R-L: 4 per hit × 5 hits × combo multiplier.

The multiplier compounds. This is why the automation must chain.

Timing

Between combo inputs: ~80ms. Longer gaps reset the chain.

Automation

attack.lua fires chains via doChainedStrike:

```
for i = 1, count do
    if i % 2 == 0 then U.m2() else U.m1() end
    if i < count then task.wait(gap) end
end
```

Gap: 0.11s. Chain count: 3–5 depending on state.

Verification

After a chain, waits 0.35s, checks HP difference:

```
local hit = t.hm.Health < hpBefore
table.insert(St.rHt, hit)
```

Hit rate is tracked for adaptive interval adjustment.

Connections

- [Chapter 19 — Attack](#) implements chaining
 - [Chapter 24 — Combat System \(PS2\)](#) covers the base
-

Chapter 27 — Global Cooldown

Learning Objectives

- Understand the v0.140 cooldown change
- Know why per-skill cooldowns are obsolete
- Recognize the implication for rotation

Before v0.140

Each skill had its own cooldown. Fire $Z \rightarrow Z$ is on cooldown. Fire $X \rightarrow X$ on cooldown.

Rotation: fire the next available skill.

After v0.140

One shared cooldown. Fire Z → all skills on cooldown for the same window.

Rotation: fire one skill, wait for GCD, fire another.

Practical Implication

The old `St.skCd[]` array is dead. There's no per-skill timing anymore.

New model: `St.gcdUntil` is a single timestamp. `fireSkill` checks:

```
if now < St.gcdUntil then return false end
-- fire
St.gcdUntil = now + Cfg.GCDWindow
```

GCD Window

Default: 1.10s. This is the approximate observed cooldown between skill casts.

Rotation Strategy

Since only one skill per window, prioritize:

1. Ultimate-tier skills when high-value window
2. Non-ultimate skills otherwise
3. Match boss state (stunned → ultimate, blocking → break)

Combo Engine

The combo engine picks the next skill from the current order. Since GCD gates all skills, only one fires per window.

Effect: skills fire in the order specified by the permutation, at GCD cadence.

Connections

- [Chapter 19 — Attack](#) implements the combo engine
- [Chapter 28 — Skills & Breathings](#) covers resource model

Chapter 28 — Skills & Breathings

Learning Objectives

- Understand the skill slots
- Know the breath resource
- Recognize the breathing activation

Skill Slots

Six slots: F, Z, X, C, V, B.

F is either a skill or block. Determined by probing at boot:

```
U.keyDown("F")
task.wait(0.15)
local isBlocking = character:GetAttribute("IsBlocking")
U.keyUp("F")
```

If `IsBlocking` true during probe, F is block. Otherwise F is a skill.

Breath Resource

Skills consume breath. Breath regenerates over time.

Reading breath:

```
local breath = humanoid:GetAttribute("Breath")
local maxBreath = humanoid:GetAttribute("MaxBreath")
local frac = breath / maxBreath
```

If `frac < 0.12`, skills are gated (except in emergency mode).

Breath regenerates. Value unknown exactly, but a few seconds between casts is usually enough.

Breathing Activation

Pressing L activates the current breathing style. This is a persistent buff that boosts damage and unlock skills.

Automation presses L every 2.5s to maintain activation.

Mastery

Mastery is a per-skill and per-weapon progression. Killing bosses grants mastery to whatever was used.

Mastery affects damage scaling. It's tracked by the game, not by our automation.

Connections

- [Chapter 27 — Global Cooldown](#) covers timing
 - [Chapter 19 — Attack](#) fires skills
-

Chapter 29 — Blood Demon Arts

Learning Objectives

- Know the BDA list
- Understand BDA vs breathing
- Recognize the swap mechanic

What BDAs Are

Blood Demon Arts are the demon-side equivalent of breathing styles. A player who becomes a demon (via Muzan's Blood) can learn a BDA.

BDA List

- Blood Manipulation
- Ice Manipulation
- Arrow Manipulation
- Shockwave
- Explosive Blood
- Dream Manipulation
- Reaper
- Obi Manipulation
- Tamari Manipulation

BDA vs Breathing

- Only one active at a time
- Breathing skills and BDA skills are the same slots (Z, X, C, V, B)
- Firing skills uses whichever is active

Swap Mechanic

Swapping between breathing and BDA requires going to a trainer. Not doable mid-combat.

For Automation

The automation doesn't care which is active. It fires skills via the same slots.

Connections

- [Chapter 28 — Skills & Breathings](#) covers skills

Chapter 30 — Boss Roster

Learning Objectives

- Know the endgame bosses
- Understand region grouping
- Recognize boss-specific quirks

Endgame Bosses

Boss	Region	Notes
Sumari	Bamboo Grove	Night-only
Yahari	Bamboo Grove	Night-only
Reaper	Bamboo Grove + Hidden Mist	Two spawns
Gyutai	Hidden Mist	
Domae	Hidden Mist	Ice boss
Datai	Nightfall	Behind red gate
Akazo	Nightfall	Windy Peak
Nezura	Deep Caves	Below cliff
Enru	Deep Caves	Underground

In-Game Display Names

Wiki Name	In-Game Name
Zoku	Zuko
Enmu	Enru
Daki	Datai
Akaza	Akazo
Douma	Domae
Gyutaro	Gyutai

Multiple names for the same boss. Both should be in `L.bosses`.

Boss Drop Preview

Hovering a boss shows a preview of drops. This is how we discovered the World Events Chest contents.

Boss HP

Boss	HP (solo)
Mother Bear	525
Zuko	250
Sabito (mini)	600
Endgame	3000+

Party scaling: +1000 per additional player.

Attack Patterns

Each boss has unique attacks. Known patterns:

- **Giyu** — Constant Flux, 3-hit slash + lunge
- **Sabito** — Water Wheel air spin
- **Yahaba** — Arrow Spikes (ground spikes)
- **Douma** — Barren Hanging Garden (ice storm), Freezing Clouds (grab)

Full pattern documentation is incomplete.

Connections

- [Chapter 24 — Combat System \(PS2\)](#) covers combat
 - [Chapter 25 — Blocking](#) notes AoE bosses
-

Part V — Detection and Evasion

Chapter 31 — Anti-Cheat Ecosystem

Learning Objectives

- Understand the layers of anti-cheat
- Know what each layer monitors
- Recognize that some layers cannot be evaded

Layers

Layer 1 — Roblox server-side validation.

The game server owns all authoritative state. It validates client reports of position, actions, and effects. This layer cannot be evaded by client-side code — the server simply decides what's true.

Layer 2 — Roblox platform anti-cheat (Hyperion / Byfron).

Kernel-level protection. Monitors memory injection, DLL loading, and known bytecode signatures. Detects the executor itself, not the script it runs.

Layer 3 — Game-specific client-side anti-cheat.

Whatever the game developers have implemented. May include honeypots, animation checks, interaction rate monitors.

Layer 4 — Manual review.

Human moderators reviewing reports, logs, and gameplay footage.

Detection Categories

Static detection — signature-based. Specific code patterns are flagged. Evade by changing code shape.

Behavioral detection — pattern-based. Actions that deviate from human norms are flagged. Evade by slowing down.

Statistical detection — outlier-based. Player is in the top 0.1% of farm rates. Evade by farming less.

Honeypot detection — trap-based. Fake targets planted to catch automation. Evade by whitelist-only.

Server validation — authoritative. Server rejects or corrects. Cannot evade.

The Realistic View

Layer 1 (server validation) determines what's physically possible.

Layers 2–4 determine what consequences you face.

You can try to stay below detection thresholds in layers 3–4. You cannot escape layer 1 or fully escape layer 2.

Connections

- [Chapter 32 — Roblox Hyperion](#) covers layer 2
- [Chapter 33 — Game-Specific Anti-Cheat](#) covers layer 3
- [Chapter 34 — The Honeypot Incident](#) documents a layer 3 case

Chapter 32 — Roblox Hyperion

Learning Objectives

- Understand what Hyperion does

- Know what it detects
- Recognize the ban mechanics

What Hyperion Is

Formerly Byfron, acquired by Roblox in 2022, deployed in 2023. Kernel-level anti-cheat.

Kernel-level means it runs with the same privileges as the operating system, below the Roblox process itself. It can:

- Inspect memory
- Read process handles
- Monitor system calls
- Detect injected code

What It Detects

- **Memory injection** — any external process modifying Roblox memory
- **Unsigned DLL loading** — most executors
- **Bytecode signatures** — specific known exploit patterns
- **Exploit frameworks** — Xeno, Synapse, Fluxus, etc. (as identities, not just their output)

Enforcement

When Hyperion detects exploitation, it reports to Roblox's server. The response escalates:

1. First detection: kick
2. Repeated: temporary ban
3. Persistent: permanent ban
4. Severe cases: hardware ID ban

The `Error 267 · Exploiting` kick is a Hyperion-driven enforcement signal.

Executor Response

Executors have responded with varying success:

- **Xeno** — still detected, uses bytecode overwriting
- **Synapse** — paid, historically better at evasion
- **Solara** — newer, still under active development
- **Script-Ware** — discontinued

The cat-and-mouse continues.

Implication for Users

If you run an executor, you are detectable at Hyperion's layer. The question is only whether detection occurs during your session. Long-term safety is not achievable.

Connections

- [Chapter 31 — Anti-Cheat Ecosystem](#) covers layers
 - [Chapter 33 — Game-Specific Anti-Cheat](#) covers layer 3
-

Chapter 33 — Game-Specific Anti-Cheat

Learning Objectives

- Understand what the game itself monitors
- Know the honeypot mechanism
- Recognize interaction-rate signatures

What the Game Monitors

From community research and our own incidents:

- **Position delta per tick** — too fast → suspicious
- **Interaction distance** — beyond `MaxActivationDistance` → flagged
- **Interaction rate** — too fast → suspicious
- **Animation mismatch** — client animations not matching server state
- **RemoteEvent fire rate** — too many per second → flagged
- **Honeypot triggers** — any interaction with planted decoys → confirmed bot

The Honeypot Mechanism

Honeypots are decoy instances placed in the world to catch automation:

1. Named with bait keywords (`BookGrimore9_Plane` — contains "ore")
2. Placed in clusters far from player (68–100 studs)
3. Cloned many times (10+ identical instances)
4. Any interaction → server-side flag

Real players never touch these. Any client that does is automation.

Interaction Rate Signatures

Legitimate human interaction rate:

- ~1 interaction per 2–5 seconds
- Movement between interactions

- Some interactions fail (misclicks, distance)

Bot interaction rate:

- ~1 interaction per 0.5 seconds
- No movement between
- All interactions succeed

The game's server logs the difference.

The 267 Kick

Error 267 · Exploiting is Roblox's platform-level enforcement code. It's not the game directly banning — the game reports a flag, and Roblox applies the enforcement.

When you see this, the account is done for that game. Potentially for all games with shared reporting.

Connections

- [Chapter 34 — The Honeypot Incident](#) covers our case study
 - [Chapter 35 — Detection Vectors](#) enumerates what to avoid
-

Chapter 34 — The Honeypot Incident

Learning Objectives

- Understand what happened
- Know the root cause
- Recognize the fix

What Happened

On 2026-10-03, the user's account was kicked with Error 267 · Exploiting .

Log Evidence

The F9 log showed:

```
[11:42:26] [Dingus][Chest] 14 targets within 100 studs:
[11:42:26] [loot] Demonic Lantern @1
[11:42:26] [loot] Lost Mask @2
[11:42:26] [loot] Meshes/BookGrimore9_Plane.008 @3
[11:42:26] [loot] Meshes/BookGrimore9_Plane.007 @4
[11:42:26] [loot] Meshes/BookGrimore9_Plane.011 @5
[11:42:26] [loot] Meshes/BookGrimore9_Plane.008 @68
```

```
[11:42:26] [loot] Meshes/BookGrimore9_Plane.007 @69
[11:42:26] [loot] Meshes/BookGrimore9_Plane.011 @75
[11:42:26] [loot] Meshes/BookGrimore9_Plane.007 @76
[11:42:26] [loot] Meshes/BookGrimore9_Plane.008 @76
[11:42:26] [loot] Meshes/BookGrimore9_Plane.007 @100
[11:42:26] [loot] Meshes/BookGrimore9_Plane.011 @100
[11:42:26] [loot] Meshes/BookGrimore9_Plane.008 @100
```

Twelve identical names. Two real items at 1–2 studs. Then the honeypot cluster at 68–100.

Root Cause Chain

1. **Keyword matching.** `ChestLootKeywords` contained `"ore"`.
`string.find("Meshes/BookGrimore9_Plane.008", "ore")` matched.
2. **No reach limit.** `U.tap("T")` fired regardless of distance.
3. **No rate limit.** Interactions fired in immediate succession.
4. **No cluster detection.** Twelve clones weren't recognized as a signature.

Server Side

The game saw the following:

- Player at position X
- 12 `RemoteEvent:FireServer` calls within 3 seconds
- All targets beyond `MaxActivationDistance`
- All targets sharing a base name pattern
- No movement between interactions

This is not human behavior. It's a confirmed automation.

Consequence

Platform-level enforcement. Account banned.

Fix

`chest.lua v4:`

- Exact-name whitelist replaces keyword matching
- 4-stud reach limit
- 10/min rate cap
- 40/session cap
- Honeypot cluster detection (≥ 3 clones $\rightarrow$ refuse)
- Reject signatures (/ , \ , Book , Grimore)

Lessons

1. **Substring matching against world instance names is unsafe.** Names can be crafted by the developers to match common exploit keywords.
2. **Interaction distance is a detection vector.** Real players cannot interact beyond prompt range.
3. **Interaction rate is a detection vector.** Real players take seconds, not milliseconds.
4. **Cluster of identical instances is a trap signature.** Real loot drops don't spawn 12 clones.
5. **The account is not recoverable.** Appeals confirm intent and can extend enforcement.

Connections

- [Chapter 17 — Chest](#) documents the fix
 - [Chapter 33 — Game-Specific Anti-Cheat](#) covers the detection layer
 - [Chapter 46 — Why Keyword Matching Fails](#) extends the analysis
-

Chapter 35 — Detection Vectors

Learning Objectives

- Enumerate what gets you flagged
- Know which are fixable
- Recognize which are not

Enumerated Vectors

V1 — Position jump per tick.

Server validates distance per tick. Too fast → flagged.

V2 — Interaction distance.

Prompt fired beyond `MaxActivationDistance` → flagged.

V3 — Interaction rate.

Too many interactions per second → flagged.

V4 — BodyMover presence.

Anti-cheat scans HRP children for `BodyVelocity`, `BodyGyro`, etc.

V5 — Sustained air time.

Player in Air for too long without legitimate reason → flagged.

V6 — Honeypot interaction.

Any interaction with a planted decoy → confirmed bot.

V7 — Animation mismatch.

Client animations not matching server-expected state.

V8 — RemoteEvent fire rate.

Too many fires per second → flagged.

V9 — Account age.

Fresh accounts more likely to be flagged on detection.

V10 — Honeypot RemoteEvent fires.

Fake remotes planted to catch scanners.

Fixable vs Not

Vector	Fixable client-side?
V1	Partially — cap speed
V2	Yes — reach limit
V3	Yes — rate limit
V4	Yes — Head parent (evades some scans)
V5	No — inherent
V6	Yes — whitelist
V7	Partially — match animations
V8	Yes — rate limit
V9	No — account-level
V10	Yes — don't scan remotes

The Pattern

Client-side evasion works on:

- Interaction distance
- Interaction rate
- Honeypot avoidance
- RemoteEvent rates

Client-side evasion doesn't work on:

- Position validation
- Sustained air time
- Account age

Implication

Every evasion is an arms race. The game adds a check, we add a counter. But some checks (server-authoritative ones) cannot be countered.

Connections

- [Chapter 33 — Game-Specific Anti-Cheat](#) covers detection
 - [Chapter 34 — The Honeytrap Incident](#) is a case study
 - [Chapter 36 — Evasion Patterns](#) covers counters
-

Chapter 36 — Evasion Patterns

Learning Objectives

- Know the standard evasion patterns
- Understand tradeoffs
- Recognize which patterns are detection-prone

Pattern 1 — Jitter

Add randomness to any pattern. Teleports, click timings, interaction sequences.

```
Teleport destination = target + random_jitter_vector
```

Prevents detection of exact-repeat patterns.

Pattern 2 — Rate Limiting

Enforce minimum gaps between actions.

```
if now - lastAction < minGap then return end
```

Prevents detection of machine-perfect cadence.

Pattern 3 — Blending

Mix automation with legitimate actions.

```
if math.random() < blendChance then  
    U.hum():Move(random_direction)  
end
```

Prevents detection of "never walks" pattern.

Pattern 4 — Bounded Exposure

Cap total actions per session.

```
if sessionActions >= sessionCap then
  killswitch()
end
```

Prevents accumulation of suspicious activity.

Pattern 5 — Cluster Avoidance

Refuse clusters of identical instances.

```
if #clones_of_same_base_name >= 3 then
  refuse_all()
end
```

Prevents honeypot triggers.

Pattern 6 — Reach Limiting

Never interact beyond a plausible distance.

```
if distance > 4 then return end
```

Prevents detection via interaction distance.

Pattern 7 — Conditional Pausing

Pause when "watched."

```
if player_nearby or suspicious_conditions then
  pause()
end
```

Prevents detection during active monitoring.

The Tradeoff

Every pattern reduces effectiveness. Jitter slows travel. Rate limits slow farming. Bounded exposure stops sessions.

The optimal balance depends on detection levels and risk tolerance.

Connections

- [Chapter 35 — Detection Vectors](#) covers what to evade

- [Chapter 33 — Game-Specific Anti-Cheat](#) covers the enemy
-

Part VI — Operational

Chapter 37 — Session Lifecycle

Learning Objectives

- Know how to start, run, and end a session
- Recognize warning signs
- Know when to stop

Start

1. Launch Roblox → Project Slayers 2
2. Wait for character spawn
3. Inject executor
4. Run loader loadstring
5. Verify boot in F9
6. Check `Chest.stats().learnMode` state
7. Verify whitelist size > 0
8. Start combat

Mid-Session Checks (every 10–15 min)

```
_G.Atk.telemetry()      -- combat state
_G.Chest.stats()        -- chest state, session count
_G.Detect.stats()       -- how many bosses detected
_G.St.fps               -- performance
```

Warning Signs

- `chest.stats().killed == true` — session cap tripped
- `chest.stats().sessionCount > 30` — approaching cap
- `detect.stats().priorityFails > 10` — priority filter unreliable
- `attack.telemetry().state` stuck on same value for >30s
- Sustained FPS < 30

End

1. Stop combat (`_G.St.cbt = false`)
2. Save config (`_G.Cfg.save("default")`)
3. Note any issues in F9

4. Unload script if available
5. Close Roblox

Cool-Down Period

Between sessions, wait at least 15 minutes. This gives:

- Detection system time to decay any flags
- You time to review the session log
- Clear separation between sessions

After a Detection

If any of these appear:

- Kick with error code
- Sudden lag
- Unusual game state
- Server message

Stop immediately. Do not rejoin on same account. Do not rejoin on same network. Wait 72+ hours before touching that account again.

Connections

- [Chapter 38 — Learn Mode Workflow](#) covers the chest setup
 - [Chapter 40 — Emergency Procedures](#) covers recovery
-

Chapter 38 — Learn Mode Workflow

Learning Objectives

- Know how to safely build a chest whitelist
- Recognize honeypot names
- Understand the arming step

Phase 1 — Learn

Load the script. Chest module is in learn mode by default.

Play normally for 10–20 minutes. Kill bosses, open chests manually, move around.

Check observations:

```
_G.Chest.listSeen()
```

Phase 2 — Identify

Look at the observations. For each name:

- Has a slash? Honeypot. Ignore.
- Contains Grimore , Book , Plane ? Honeypot. Ignore.
- Has trailing .NNN ? Clone signature. Ignore.
- Is a real item you saw in-game? Candidate.

Phase 3 — Approve

```
_G.Chest.approve("World Events Chest")  
_G.Chest.approve("Common Chest")  
_G.Chest.approve("Gold Coin")
```

Each call prints confirmation.

Phase 4 — Save

```
_G.Cfg.save("default")
```

Whitelist persists if ChestWhitelist is in PERSIST list.

Phase 5 — Arm

```
_G.Chest.setLearnMode(false)
```

Now the module fires on approved names only.

Phase 6 — Verify

Walk near a boss, kill it, watch F9:

```
[Dingus] killed X (1)  
[Dingus][Chest] boss died - waiting 5.2s then scanning corpse  
[Dingus][Chest] 3 items at corpse - attempting exactly 3  
[Dingus][Chest] collected X (attempt 1/3)  
[Dingus][Chest] corpse cycle done · 3/3 collected · moving to next boss
```

Adding New Items

If you encounter a real item not collected:

1. Note the exact name
2. Verify it's not a honeypot
3. `_G.Chest.approve("Name")`
4. `_G.Cfg.save("default")`

Removing Bad Items

If an approved name turns out to be a trap:

```
_G.Chest.unapprove("Name")  
_G.Chest.resetKillswitch()
```

Connections

- [Chapter 17 — Chest](#) covers the module
 - [Chapter 40 — Emergency Procedures](#) covers recovery
-

Chapter 39 — Whitelist Management

Learning Objectives

- Understand the whitelist lifecycle
- Know when to add/remove
- Recognize when the whitelist is bad

Lifecycle

Whitelists start empty. Grow during learn mode. Get pruned when traps are identified. Stabilize over time.

A mature whitelist has 5–15 real item names.

When to Add

- You saw a real item in-game
- The scanner isn't collecting it
- You verified the name

When to Remove

- The item no longer drops
- The name turned out to be a honeypot
- You want to reduce exposure

When to Reset Entirely

If the whitelist has more than 3 entries you're unsure about, reset and relearn. Bad whitelists are worse than empty whitelists.

```
_G.Cfg.ChestWhitelist = {}  
_G.Cfg.save("default")
```

Sharing Whitelists

Do not share whitelists. Another user's whitelist may contain traps that only apply to their game state.

Each player should build their own whitelist from their own observations.

Connections

- [Chapter 17 — Chest](#) covers the module
 - [Chapter 38 — Learn Mode Workflow](#) covers the workflow
-

Chapter 40 — Emergency Procedures

Learning Objectives

- Know how to respond to each failure mode
- Recognize when to stop
- Understand irreversible consequences

Panic Hide GUI

RightCtrl + Backspace. Detaches GUI.

Killswitch Triggered

If `_G.Chest.stats().killed` is true:

1. Do not reset immediately
2. Read F9 for what triggered it
3. If session cap: keep playing, wait until next session
4. If honeypot: review whitelist, unapprove anything suspicious, then reset

Combat Completely Dead

```
_G.Atk.forceStopRetreat()
_G.Atk.abortLoot()
_G.St.tgt = nil
_G.St.cbt = false
task.wait(0.5)
_G.St.cbt = true
```

If still dead:

```
_G.Detect.invalidate()
_G.Detect.dump() -- check this returns non-zero humanoids
```

If `dump()` returns 0 humanoids, the walk is broken. Reload.

Roblox Kicked

If you see `Error 267 · Exploiting :`

1. Do not rejoin on that account
2. Close Roblox fully
3. Wait 72 hours minimum
4. Use a different account on a different network for further testing
5. Do not appeal

Everything Wrong

```
_G.St.cbt = false
_G.Chest.setLearnMode(true)
for k in pairs(_G.Cfg.SpoofMethods) do
    _G.Cfg.SpoofMethods[k] = false
end
_G.Cfg.save("default")
```

Then close Roblox. 24-hour cool-down minimum.

Recovery From Ban

There is no recovery. Error 267 is permanent for the affected account. Do not appeal. Move on.

Connections

- [Chapter 34 — The Honeypot Incident](#) is a case study
 - [Chapter 37 — Session Lifecycle](#) covers normal flow
-

Part VII — Incident Post-Mortems

Chapter 41 — Post-Mortem: Honeypot Ban

What Happened

Account kicked with Error 267 after chest module fired prompts on honeypot decoys.

Why

Keyword-based matching. `ore` matched `Grimore`. Reach limit absent. Rate limit absent. Cluster detection absent.

Impact

Permanent account ban.

Root Cause

Design assumption: "If it matches a loot keyword, it's loot."

Reality: The game developers plant decoys with bait names.

Fix

`chest.lua` v4: whitelist-only, reach limit, rate limit, cluster detection, reject signatures.

Prevention

- Never keyword-match world instance names
- Always verify reach before firing
- Always rate-limit interactions
- Always detect cluster traps

Pattern

The Lesson: names are attacker-controlled. Any name-based logic must be defensive.

Connections

- [Chapter 17 — Chest](#) covers the fixed module
- [Chapter 34 — The Honeypot Incident](#) is the source

Chapter 42 — Post-Mortem: Retreat Loop

What Happened

Combat remained in RETREAT state indefinitely. Attacked nothing.

Why

`RetreatHP = 0.55` and `RetreatClearHP = 0.75`. After 3 boss hits, HP dropped to 49%. Retreat fired. HP never climbed above 75%. Retreat refired every cycle.

Impact

Combat dead. User reported "it attacks nothing."

Root Cause

Retreat threshold too high. Exit threshold unreachable without healing.

Fix

v21: `RetreatHP = 0.30`, `RetreatClearHP = 0.55`, cooldown 8s, watchdog 6s.

Prevention

- Every state must have a reachable exit condition
- Watchdog on any state that could lock
- Log state transitions for diagnosability

Pattern

The Lesson: if a state has an exit condition based on external recovery (healing, resource regen), assume recovery may not happen. Add a cooldown or watchdog.

Connections

- [Chapter 19 — Attack](#) covers the fixed module

Chapter 43 — Post-Mortem: Fly Rejection

What Happened

Fly engaged but character didn't move as expected. Server repeatedly repositioned player.

Why

Client wrote `BodyVelocity = 85 studs/s`. Server observed actual velocity of 2–31 studs/s. Server reconciliation pulled player back every ~1 second.

Impact

No effective movement. Rub-berbanding. One instance dropped player into baseplate.

Root Cause

Server-side position validation rejected client-side physics. BodyMover writes are inputs, not authoritative.

Fix

Retired fly as primary movement. Teleport-chase adopted (CFrame writes plus jitter plus velocity reset).

Prevention

- Test detection surface before building on a technique
- Assume server reconciliation for any movement
- Have a fallback for any movement technique

Pattern

The Lesson: any physics-based movement (BodyVelocity, BodyPosition) is subject to server reconciliation. Test the effective speed before building on it.

Connections

- [Chapter 24 — Combat System \(PS2\)](#) covers movement mechanics
 - [Chapter 47 — Why Fly is Fundamentally Detectable](#) extends the analysis
-

Chapter 44 — Post-Mortem: Detection Zero

What Happened

Boss scan returned zero humanoids on every tick. Combat had no targets.

Why

v6 introduced per-iteration throttle. After 22ms, every stack pop triggered `task.wait(0.005)`. Walk was throttled to ~200 iterations/second.

Impact

40,000-instance workspace required 3+ minutes per scan. Scan returned empty within budget.

Root Cause

Regression in walker design. v5 yielded every 2,000 iterations; v6 yielded every iteration past a time budget.

Fix

v7 batched yields every 500 iterations. v8 multi-container BFS with per-root budgets.

Prevention

- Yield at iteration boundaries, not time boundaries
- Test walker on realistic workspace before shipping
- Diagnostic per-root stats

Pattern

The Lesson: throttling is a subtle killer. Time-budgeted throttle $\neq$ iteration-budgeted yield.

Connections

- [Chapter 14 — Detect](#) covers the fixed module
-

Chapter 45 — Post-Mortem: Hotbar Contention

What Happened

Crow tool and weapon repeatedly unequipped each other. Combat had no weapon equipped.

Why

Quests tapped hotbar slot 5 for crow. Attack tapped slot 3 for weapon 200ms later. Each stomped the other.

Impact

Crow never stayed equipped. Weapon never stayed equipped. Combat ineffective.

Root Cause

Two subsystems accessing shared resource without coordination.

Fix

`hotbar.lua` mutex. All hotbar taps go through `H.tap(holder, key)`. Mutual exclusion.

Prevention

- Any shared resource accessed by multiple subsystems needs a lock
- Timeout on locks to prevent stuck mutexes
- State inspection for diagnostics

Pattern

The Lesson: shared resources are a coordination problem. When two writers touch the same key, one always wins. Only a lock makes both succeed.

Connections

- [Chapter 16 — Hotbar](#) covers the mutex
 - [Chapter 18 — Quests](#) acquires it for crow
 - [Chapter 19 — Attack](#) acquires it for weapons
-

Part VIII — Deeper Analysis

Chapter 46 — Why Keyword Matching Fails

The General Problem

Any name-based logic in an adversarial environment is vulnerable:

- Attacker (game developer) controls the names
- Defender (script) tries to guess names
- Attacker wins by choosing names that match defender's keywords

The Specific Case

`ChestLootKeywords` contained `"ore"`. Game developers named a decoy `BookGrimore9_Plane`. `"ore"` matched `"Grimore"`.

The Pattern

Defender keyword	Attacker bait name	Result
ore	BookGrimore9	Match, fire on decoy
coin	CoinPurseDecoy	Match, fire on decoy
drop	BackdropProp	Match, fire on decoy
loot	LootCrateProp	Match, fire on decoy

Any keyword can be embedded in a decoy name.

The Fix

Stop guessing. Use exact names of verified items. Build the list from observation, not inference.

Generalization

This applies to:

- Boss detection (name keywords)
- Crow detection (name keywords)
- Weapon detection (name keywords)
- Skill detection (name keywords)

Every keyword list is potentially exploitable. The safest lists are the shortest and most specific.

The Tradeoff

Short lists don't catch new items. Long lists catch decoys.

Balance: relearn frequently, prune aggressively, never guess.

Connections

- [Chapter 34 — The Honeypot Incident](#) is the case study
 - [Chapter 17 — Chest](#) is the fix
-

Chapter 47 — Why Fly is Fundamentally Detectable

The Argument

Fly requires the player to move faster than the game's physics allow. Server-side, this is either:

1. Accepted → server trusts client physics (would break every game)
2. Rejected → server corrects position (kicks or repositions)

Roblox uses model 2 for FilteredEnabled games. Therefore, fly is:

- Rejected by server (position validation)
- Corrected regularly (repositioned to valid location)
- Detectable by rate of corrections

The Evidence

- Client wrote 85 studs/s. Server observed 2–31 studs/s.
- Server reconciliation pulled player back every ~1 second.
- One attempt dropped player into baseplate (CFrame NaN).

The Implication

No client-side physics manipulation is undetectable in a FilteredEnabled game. Speed, fly, noclip — all subject to server validation.

The Alternative

Use movement techniques that mimic legitimate motion:

- Teleport-chase (CFrame writes within tolerance)
- Speed up WalkSpeed moderately (2× max)
- Ground-based movement with high RunSpeed

These are all detected in some cases. But the detection surface is smaller.

The Deeper Point

Every game has a maximum legitimate speed. Exceeding it is detectable. The only question is whether the detection threshold is high enough to accommodate your speed.

Connections

- [Chapter 43 — Post-Mortem: Fly Rejection](#) covers our specific case
 - [Chapter 31 — Anti-Cheat Ecosystem](#) covers layers
-

Chapter 48 — The Copycat Problem

The Problem

Multiple scripts for the same game share common patterns:

- Same keyword lists
- Same movement techniques
- Same chest detection methods

When one script gets caught, the game's anti-cheat updates to catch the pattern. All scripts using that pattern get caught.

The Example

Chest keyword matching. If 10 scripts all use "ore" as a keyword, the game names one decoy `BookGrimore9_Plane` and catches all 10.

The Implication

You can't rely on pattern-matching as a defense. If another script gets caught, your script is next.

The Response

Use behaviorally distinct patterns:

- Unique timing (not the common 0.5s)
- Unique distances (not the common 5-stud)
- Unique thresholds (not the common 30%)

Or use whitelist-based detection (unique per user).

The Deeper Point

In a mass-adoption scenario, scripts become homogeneous and therefore detectable.

The Realistic View

Nothing you do makes you unique against a determined anti-cheat. But being less homogeneous than the median script raises the threshold slightly.

Connections

- [Chapter 35 — Detection Vectors](#) covers what gets flagged
 - [Chapter 36 — Evasion Patterns](#) covers counters
-

Chapter 49 — Ethics of Game Automation

Framing

This is not a lecture. It's an honest map of the ethical terrain.

The Case Against

Against the game developer:

- Automation distorts engagement metrics
- Bots inflate the economy, deflating player-earned value
- Developers must spend money on anti-cheat instead of features

Against other players:

- In shared-economy games, bots increase resource supply

- Leaderboards and achievements lose meaning
- PvP-adjacent mechanics (bounty boards, faction wars) become unfair

Against the platform:

- Roblox ToS violation
- Enforcement resource expenditure

The Case For

Personal freedom:

- You can do what you want with your own account
- Using automation on private servers is arguably fine

Research value:

- Understanding anti-cheat systems has defensive value
- Documenting the state of the art helps others make informed decisions

Accessibility:

- Some players use automation to compensate for disability
- Some players use it to save time on repetitive tasks

The Uncomfortable Middle

Most users of scripts like Dingus-Slayer:

- Are not trying to be malicious
- Are not thinking about the developers
- Are just trying to save time on a game they enjoy

This is a morally gray position. Not defensible, not indefensible.

What the Project Does About It

1. **Documents the risk clearly.** Every doc warns about bans.
2. **Doesn't monetize.** No revenue from the exploit.
3. **Restricts scope.** Boss farming only, no PvP interaction.
4. **Provides recovery procedures.** Bans are documented.

What it doesn't do:

1. Ask the user to think about the ethics
2. Try to compensate the developers
3. Prevent any use case

The Position

The project exists. Users can decide. The docs inform.

Connections

- [Chapter 4 — Why This Project Exists](#) covers motivation
 - [Chapter 50 — Legal Landscape](#) covers the legal frame
-

Chapter 50 — Legal Landscape

Roblox ToS

Explicitly prohibits:

- Third-party software
- Automation of gameplay
- Circumvention of anti-cheat

Enforcement: account termination, hardware ban, potential legal action for commercial-scale operations.

DMCA

If the game's assets or code were copied in the project (they weren't — our code is original), DMCA takedown would apply.

CFAA

Computer Fraud and Abuse Act (US). Applying it to game automation is contested. Some cases (Van Buren v. United States, 2021) narrowed scope, but the gray area remains.

GDPR / Data Protection

Our project doesn't handle personal data. No GDPR exposure.

The Project's Position

- Original code, no copied assets
- No monetization, so no commercial exposure
- Documentation disclaims warranty
- Research license available for academic use

The User's Position

- Roblox ToS: violated

- DMCA: no exposure
- CFAA: gray area
- Other laws: unknown

The Realistic View

For individual users, the practical risk is account ban. Legal action is possible but rare.

For distributors (paid script hubs), legal exposure increases. Roblox has pursued DMCA takedowns against major hubs.

Connections

- [Chapter 4 — Why This Project Exists](#) covers motivation
 - [Chapter 49 — Ethics of Game Automation](#) covers ethics
-

Part IX — Open Problems

Chapter 51 — Boss Roster Gap

The Problem

The wiki's boss list is incomplete. Secondary sources have partial lists. We've inferred some names from screenshots.

What We Have

- Endgame bosses: Sumari, Yahari, Reaper, Gyutai, Domae, Datai, Akazo, Nezura, Enru
- Some Map 1 names: Zoku, Mother Bear, Kaden, Sabito, Giyu, Slasher, Nezuko
- Some Map 2 names: Muichiro, Inosuke, Rengoku, Renpeke

What We Don't Have

- Complete HP values
- Complete drop tables
- Complete spawn conditions
- Complete attack patterns
- Complete level requirements

Impact

Detection may miss bosses whose names aren't in `L.bosses`. Target selection may prefer bosses whose drops aren't worth it.

The Fix

Re-fetch slayers2roblox.wiki/en/bosses/slayers-2-boss-guide/ . It has 12KB of content we fetched once and never analyzed.

Connections

- [Chapter 30 — Boss Roster](#) covers the known roster
 - [Chapter 13 — Lists](#) covers the data structure
-

Chapter 52 — Execute Mechanic

The Problem

Bosses below 10 HP enter a "knocked" state. Player must be above and press G. Otherwise target recovers with partial HP.

Current Behavior

Our automation ignores the knocked state. Bosses either die from continued damage or recover.

The Fix

Add to combat state machine:

- Check target HP < 10
- If knocked, teleport above target
- Press G

Implementation Notes

- Knocked state has a distinctive animation or attribute
- Player must be above target (Y position check)
- 30-second window
- Red cross indicator above target

Risk

Position teleport above target is a detectable pattern. Need to blend.

Connections

- [Chapter 24 — Combat System \(PS2\)](#) covers the mechanic
 - [Chapter 19 — Attack](#) is where the fix goes
-

Chapter 53 — Quest Progression

The Problem

We don't fully understand how crow quests progress. Do they reset on a timer? Do they accumulate? Do they affect reputation?

What We Know

- 5 quest slots
- Quests auto-assign
- When a quest target is killed, quest completes
- New quest assigned on next cycle

What We Don't Know

- Exact timer mechanism
- Whether quest failures penalize
- Whether quest completion affects rewards
- Whether quest types vary

Impact

We can't fully optimize quest farming. We can't predict quest refill timing.

The Fix

Longer observation sessions. Track quest assignments over time. Log transitions.

Connections

- [Chapter 18 — Quests](#) covers the current implementation
-

Chapter 54 — Adaptive Thresholds

The Problem

Config values are static. Attack interval, retreat thresholds, chest reach — all fixed.

The Idea

Adapt thresholds based on observed behavior:

- If retreat always exits early, raise the threshold
- If chest interactions always fail, increase reach

- If detection correlates with specific patterns, tune those

The Risk

Adaptive systems can drift into bad configurations. Need monitoring.

The Design

- Track success rate per threshold
- Adjust on a slow cadence
- Cap adjustments within reasonable ranges
- Revert if success rate drops

Impact

Higher effectiveness. More complexity.

Connections

- [Chapter 11 — Config](#) covers static thresholds
 - [Chapter 19 — Attack](#) uses retreat thresholds
-

Chapter 55 — Community Detection

The Problem

If multiple users run Dingus-Slayer in the same game session, they exhibit similar patterns. Anti-cheat could detect clusters.

The Idea

Add randomization per user install:

- Random default thresholds within ranges
- Random teleport jitter seeds
- Random interaction cadences

The Implementation

On first boot, generate a per-install random seed. Use it to vary defaults within safe ranges.

Impact

Users don't all exhibit the same patterns. Harder to detect as a script family.

Connections

- [Chapter 48 — The Copycat Problem](#) covers the general case
 - [Chapter 11 — Config](#) is where randomization would live
-

Part X — Appendices

Appendix A — Full API Reference

Loader

Entry point. Run via `loadstring(game:HttpGet(...))()`.

Not callable from within scripts.

Ctx (global)

Field	Type	Description
<code>Ctx.St</code>	table	Shared runtime state
<code>Ctx.Cfg</code>	table	Configuration
<code>Ctx.Util</code>	table	Utilities
<code>Ctx.Lists</code>	table	Data tables
<code>Ctx.Detect</code>	table	Boss/mob scanner
<code>Ctx.Scan</code>	table	Crow scanner
<code>Ctx.Hotbar</code>	table	Hotbar mutex + item DB
<code>Ctx.Spoof</code>	table	Client hardening
<code>Ctx.Chest</code>	table	Loot collection
<code>Ctx.Quest</code>	table	Crow quest routing
<code>Ctx.Atk</code>	table	Combat
<code>Ctx.Opt</code>	table	Optimizers
<code>Ctx.Gui</code>	table	GUI
<code>Ctx.Loops</code>	array	Scheduler loops
<code>Ctx.Errors</code>	array	Boot errors

Util

Function	Returns
<code>U.hum()</code>	local Humanoid or nil
<code>U.hrp()</code>	local HumanoidRootPart or nil

Function	Returns
<code>U.tap(k)</code>	press and release a key
<code>U.keyDown(k)</code>	hold a key
<code>U.keyUp(k)</code>	release a key
<code>U.m1()</code>	left click
<code>U.m2()</code>	right click
<code>U.clock()</code>	monotonic time
<code>U.xzDist(a, b)</code>	planar distance
<code>U.walkTree(root, depth, fn, yield)</code>	DFS iteration
<code>U.notify(title, text, dur)</code>	Roblox notification
<code>U.report()</code>	capability dump string

Detect

Function	Returns
<code>D.scanBosses(force, includeNonPriority)</code>	boss list
<code>D.scanMobs(force)</code>	mob list
<code>D.nearbyAll(force)</code>	all humanoids
<code>D.pickTarget()</code>	(target, kind)
<code>D.isEnemyAttacking(e)</code>	bool
<code>D.isEnemyBlocking(e)</code>	bool
<code>D.isEnemyStunned(e)</code>	bool
<code>D.updateThreats()</code>	void
<code>D.scanItems(radius)</code>	item list
<code>D.scanChests(radius)</code>	chest list
<code>D.bestItem(radius)</code>	item or nil
<code>D.bestChest(radius)</code>	chest or nil
<code>D.reportTargetFail(e)</code>	void
<code>D.clearSticky()</code>	void
<code>D.invalidate()</code>	void
<code>D.stats()</code>	stats table
<code>D.dump()</code>	diagnostic dump

Hotbar

Function	Returns
H.acquire(holder, dur)	bool
H.release(holder)	bool
H.isLocked()	holder or nil
H.tap(holder, key, wait)	bool
H.tapVerified(holder, key, expected, wait)	(bool, info)
H.equipFirstOf(holder, category, wait)	(bool, info)
H.slotOf(itemName)	key or nil
H.slotsOfCategory(cat)	array
H.hasItem(name)	bool
H.scanSlots(holder)	void
H.dumpSlots()	void
H.state()	state table

Chest

Function	Returns
Chest.scan(radius)	target list
Chest.sweep(radius)	fired count
Chest.collectAll(r, passes, deadline)	(fired, pass)
Chest.collectAtBoss(pos, name)	(fired, attempts)
Chest.collectPassive()	void
Chest.setEnabled(v)	void
Chest.setLearnMode(v)	void
Chest.approve(name)	bool
Chest.unapprove(name)	bool
Chest.listSeen()	array
Chest.dump()	array
Chest.stats()	stats table
Chest.resetCooldowns()	void
Chest.resetKillswitch()	void
Chest.abort()	void

Quest

Function	Returns
Q.readLevel()	int
Q.readHunts()	array
Q.readActiveQuests()	array
Q.isPriority(name)	bool
Q.getPriorityBosses()	array
Q.cycle()	void
Q.dumpStructure()	void
Q.stats()	stats table

Attack

Function	Returns
A.combatTick()	void (scheduler)
A.forceScan()	void
A.forceMove()	void
A.fModeInfo()	table
A.comboInfo()	table
A.resuffleCombos()	void
A.telemetry()	table
A.reprobeHoldSkills()	void
A.lootNow()	void
A.scanPromptsNow()	void
A.refreshWeapon()	void
A.abortLoot()	void
A.forceStopRetreat()	void

Spoof

Function	Returns
Sp.tick()	void (scheduler)
Sp.stats()	stats table
Sp.setMethod(name, on)	bool
Sp.listMethods()	array

Appendix B — Config Key Catalog

Combat

Key	Default	Purpose
AtkRange	8	Max attack distance
AtkInterval	0.38	Base attack interval
AtkIntMin	0.22	Minimum adaptive interval
AtkIntMax	0.65	Maximum adaptive interval
StunAtkInt	0.20	Interval during stun punish
HitWindow	15	Hit history window
RunSpeed	16	Movement speed

Retreat

Key	Default	Purpose
RetreatHP	0.30	Retreat threshold
CriticalHP	0.12	Hard retreat threshold
RetreatDelay	2.0	Retreat duration
RetreatClearHP	0.55	Retreat exit threshold
RetreatCooldown	8.0	Min gap between retreats

Scanning

Key	Default	Purpose
ScanTTL	1.2	Scan cache TTL
CrowCheckT	2.5	Crow cycle interval
QuestCycleT	6.0	Quest cycle interval

Skills

Key	Default	Purpose
SkillKeys	{"F", "Z", "X", "C", "V", "B"}	Skill slot keys
SkillUnlocked	all true	Per-slot enable

Key	Default	Purpose
RotationOrder	{2,3,4,5,6}	Skill rotation order
GCDWindow	1.10	Global cooldown

Chest

Key	Default	Purpose
ChestEnabled	true	Master enable
ChestLearnMode	true	Learn mode default
ChestOnKill	true	Post-kill cycle
ChestPassive	true	Idle sweep
ChestRadius	10	Passive scan radius
ChestBossLootRadius	15	Corpse scan radius
ChestBossLootDelayMin	3.0	Min corpse wait
ChestBossLootDelayMax	9.0	Max corpse wait
ChestBossInterItemMin	1.5	Min inter-item delay
ChestBossInterItemMax	3.0	Max inter-item delay
ChestReachDist	4	Max interaction distance
ChestMinInteractGap	3.0	Min interaction gap
ChestMaxInteractionsPerMin	10	Per-minute cap
ChestSessionInteractionCap	40	Session cap
ChestHoneypotThreshold	3	Cluster detection threshold
ChestWhitelist	{}	Approved item names

Teleport

Key	Default	Purpose
TeleportCd	0.22	Min gap between teleports
TeleportStrike	4.5	Landing distance from target
TeleportHeight	3	Y offset
TeleportJitter	2	Random destination jitter
TeleportWalkBlend	0.6	Post-teleport walk chance

Combo

Key	Default	Purpose
ComboBurst	4	Base chain count
ComboGap	0.11	Gap between clicks
ComboOrderCount	4	Distinct permutations
ComboReshuffleN	3	Rotations before reshuffle
M1MaxHz	10	Max M1 rate

Spoofers

Key	Default	Purpose
SpoofMethods	table	Per-method enable
SpoofSpeedMult	1.25	WalkSpeed multiplier
SpoofJumpMult	1.15	JumpPower multiplier
SpoofWriteHz	10	Method tick rate

GUI

Key	Default	Purpose
GuiConcealed	true	Parent to gethui
GuiPanicHide	true	Enable panic keybind

Appendix C — Glossary

Anti-cheat — Any system that detects and responds to cheating. Multiple layers: server validation, platform (Hyperion), game-specific, manual review.

BodyMover — Roblox physics instance (`BodyVelocity` , `BodyGyro` , `BodyPosition` , `BodyForce` , `LinearVelocity` , `AlignOrientation` , `AlignPosition`). Used to manipulate character physics.

Boss — In this project, an AI NPC with a specific name and HP/drop table.

CFrame — Roblox coordinate frame. Position + rotation. Used to manipulate character position.

Crow — Kasugai Crow. Player's companion that provides quests.

Detect — The `detect.lua` module. Scans for humanoids and provides targeting.

Detect walk — The process of scanning the workspace for humanoids.

Executor — Third-party tool that enables arbitrary script execution in Roblox. Examples: Xeno, Solara, SynapseZ.

FilteredEnabled — Roblox game mode where the server owns authoritative state. Client is not trusted. Default since 2016.

GCD — Global cooldown. Single shared timer for all skill casts (post v0.140).

Honeypot — A decoy instance planted by game developers to catch automation. Usually named with bait keywords, placed in clusters, far from player.

Hotbar — The 9-slot action bar at the bottom of the screen. Keys 1–9.

Hyperion / Byfron — Roblox's kernel-level anti-cheat. Monitors memory injection, DLL loading, and known exploit patterns.

Loader — The bootstrap script. Fetches modules, compiles, executes, boots main.

MaxActivationDistance — A ProximityPrompt property. Maximum distance at which the prompt can be triggered.

M1 / M2 — Left click / right click. Combat inputs.

ProximityPrompt — Roblox instance that shows an interaction prompt when the player is nearby. Used for chests.

Rate limit — A cap on actions per unit time. Prevents detection of machine-perfect cadence.

Reconciliation — Server's correction of client state. When the client reports a position the server doesn't believe, the server repositions.

Reach — Maximum interaction distance. Legitimate players can only interact within ~4 studs.

Spoofers — A method that maintains client-side state despite server corrections.

St — `Ctx.St`. Shared runtime state table.

State machine — The combat module's model of combat states. Transitions between IDLE, TELEPORT, STRIKE, etc.

Whitelist — In chest context, an explicit list of item names that can be collected. Exact-match.

Appendix D — Reading List

Community Sources

- Roblox DevForum — the primary source for technique discovery and discussion
- r/robloxexploiting — community subreddit (content varies)
- Script hub forums — historical discussion (many defunct)

Official Sources

- Roblox Creator Documentation — API reference
- Roblox Terms of Service — platform rules
- Hyperion documentation (limited) — anti-cheat details

Project Files

- README.md — architecture and module reference
- USAGE.md — operational playbook
- Code_Audit.md — all findings
- LICENSE — terms of use
- CHANGELOG.md — version history

External Wikis

- project-slayers.fandom.com — canonical PS2 wiki (incomplete)
 - slayers2roblox.wiki — secondary wiki (fetched, not analyzed)
 - allthings.how — guides for PS2 mechanics
-

Appendix E — Source Ledger

Fetches and Analyzes

- project-slayers.fandom.com/wiki/Features — combat mechanics
- project-slayers.fandom.com/wiki/Zuko — boss page
- project-slayers.fandom.com/wiki/Map_1 — regions
- allthings.how — v0.140 cooldown article
- nerdschalk.com — boss HP reference
- sportsrant.com — breathing tier list
- roonby.com — trainer locations
- gamezebo.com — demon arts list
- gameshorizon.com — BDA tiers
- bloxinformer.com — clan tier list
- techwiser.com — clan rarity table
- gamer.org — clan naming

- kongbakpao.com — Nightfall tier list
- github.com/SqIDev/Roblox-Anti-Exploit-System — anti-cheat patterns
- github.com/luvmad/VANITY-ANTICHEAT — detection methods
- github.com/tyronetheqt/Xeno — executor architecture
- devforum.roblox.com — multiple threads on anti-cheat, camera, humanoid

Fetches, Not Analyzed

- slayers2roblox.wiki/en/bosses/slayers-2-boss-guide — 12KB, evidence file missing

Located, Not Fetches

- slayers2wiki.net, slayers2wiki.wiki, slayers-2.wiki
- projects-layers2.com

Appendix F — Audit Findings by ID

Cross-reference to `Code_Audit.md`. Full list of findings:

Critical

- C-01 — Honeypot-triggered ban (FIXED)
- C-02 — Fly nil-call (FIXED)
- C-03 — Retreat loop lockout (FIXED)
- C-04 — Walk throttle (FIXED)
- C-05 — Shared iteration counter (FIXED)
- C-06 — Fly rejection (FIXED via retirement)
- C-07 — CFrame NaN (FIXED)
- C-08 — Auto-block during fly (FIXED)
- C-09 — Chest cycle stuck (FIXED)
- C-10 — Priority filter deadlock (FIXED)
- C-11 — Corrupted cache (FIXED)
- C-12 — `task.spawn` errors ignored (FIXED)
- C-13 — BodyMover scrub (FIXED)
- C-14 — `spawnTime` uninit (OPEN)

High

- H-01 to H-24 — see `Code_Audit.md`
- H-01, H-02, H-03 — attack issues (OPEN)
- H-04 to H-24 — mostly FIXED

Medium

- M-01 to M-31 — see `Code_Audit.md`
- ~6 open findings

Low

- L-01 to L-18 — see `Code_Audit.md`
- ~5 open findings

Closing Notes

This document is a study session, not a manual. It's the internal reasoning made legible. If you've read this far, you now know:

- What the project is and why
- How every module works and why it's built that way
- What the game mechanics are that shape the design
- What detection you're up against
- What has gone wrong and why
- What problems remain open

The project is not finished. It's not finished because it can't be finished — the game updates, the anti-cheat evolves, and the arms race continues. This document captures the state of that race as of 2026-10-03.

Every session you run is a data point. Every incident adds to the record. Every fix narrows the window of what's exploitable.

Be careful out there.

End of study notes. Document version 1.0. Compiled 2026-10-03.